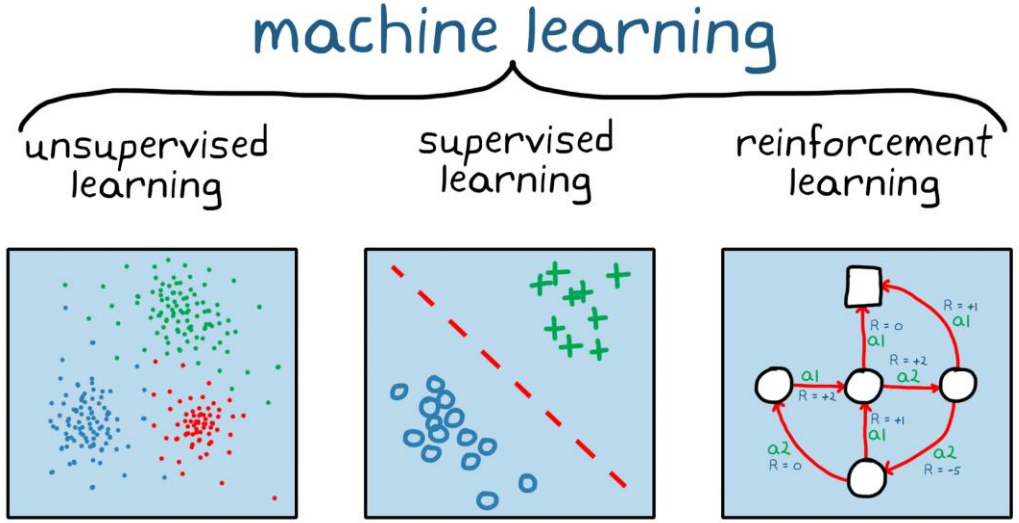
Introduction to Reinforcement Learning (RL)

Vikky Masih,
Research Scholar,
Mehta Family School of Data Science & Artificial Intelligence,
IIT Guwahati

11-12 July 2023, DS&AI Research Scholars' Discussion Group, IITG

Machine Learning (ML)

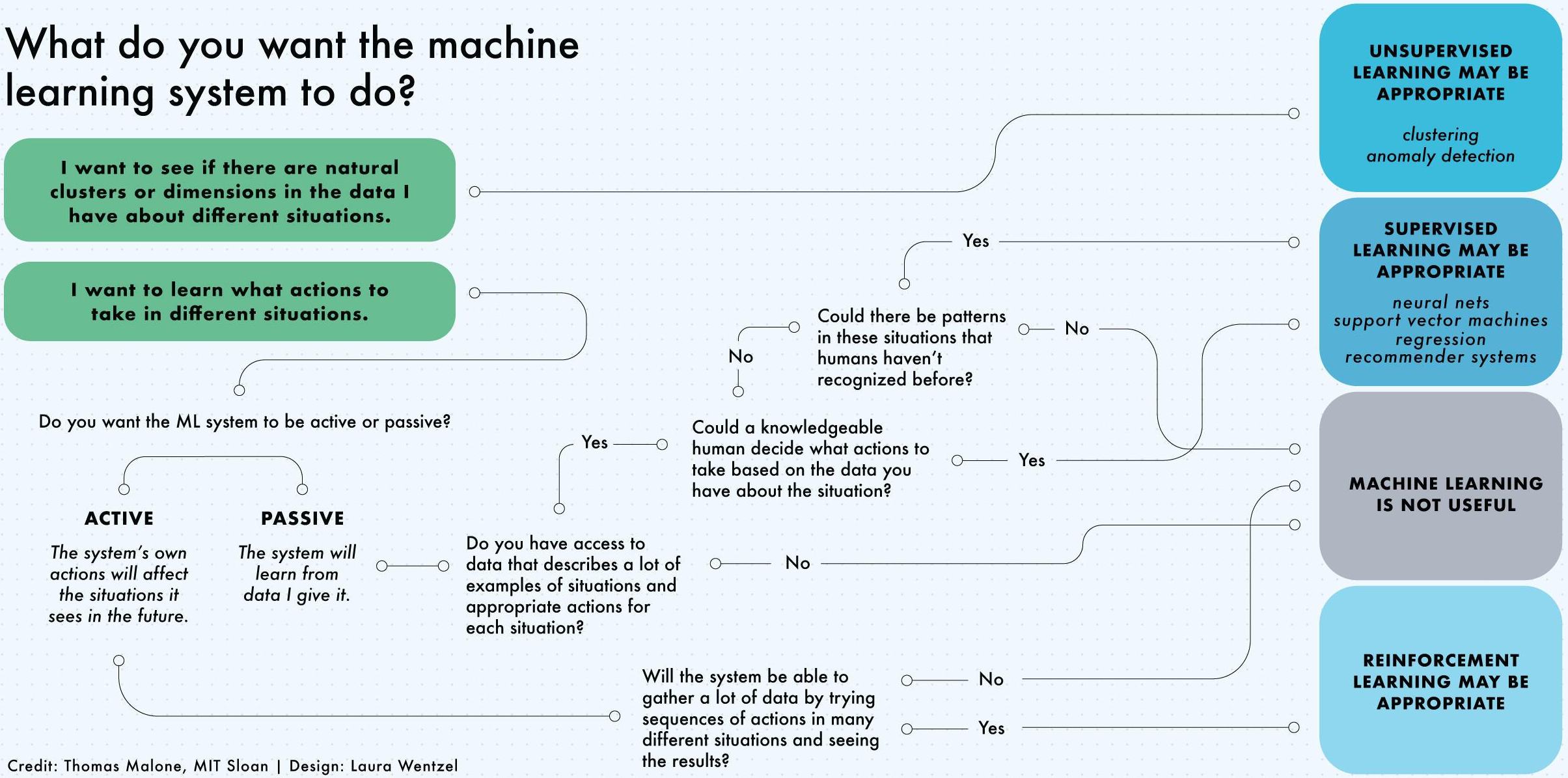
- ML is fundamental concept of AI
  - Learn to improve performance via experience



- Unsupervised Learning:
  - Clustering, Anomaly Detection, PCA, etc.
  - Find patterns in input data
- Supervised Learning:
  - Classification and Regression
  - Labelled data for training
- Reinforcement Learning:
  - Decision making under uncertainty
  - Learn to improve performance via interacting with environment

Image Credits: mathworks.com

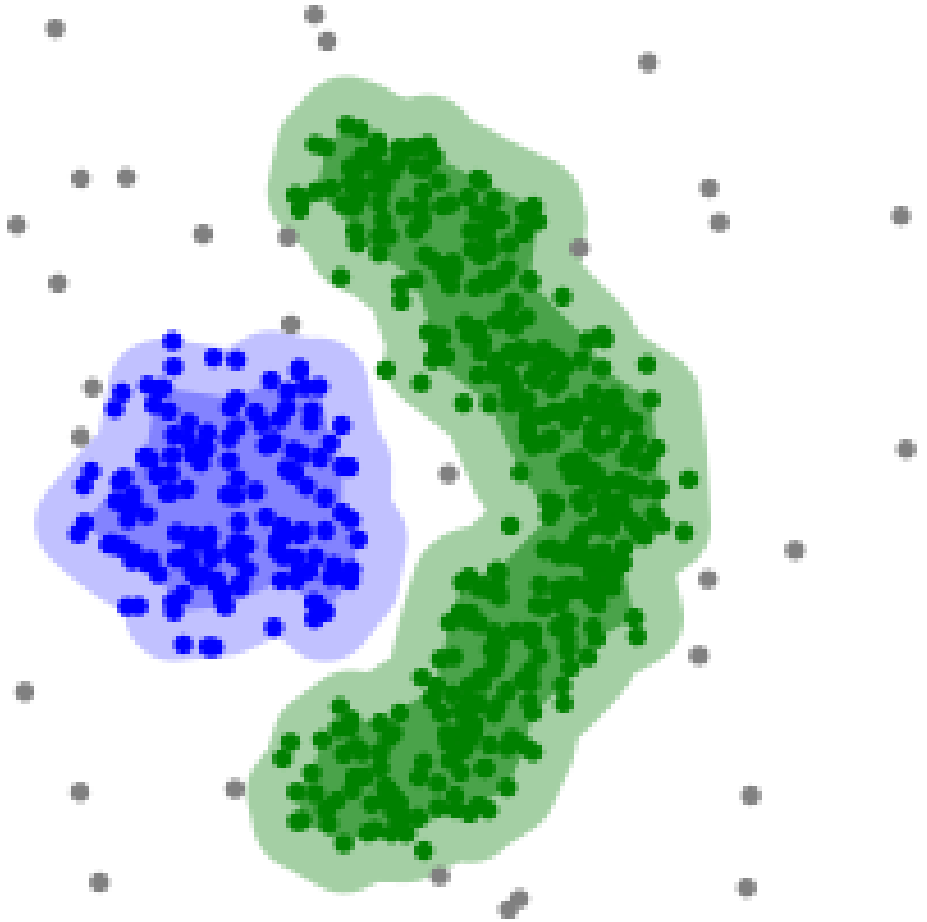
What do you want the machine learning system to do?



Credit: Thomas Malone, MIT Sloan | Design: Laura Wentzel

Ref: https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained

Machine Learning (ML)

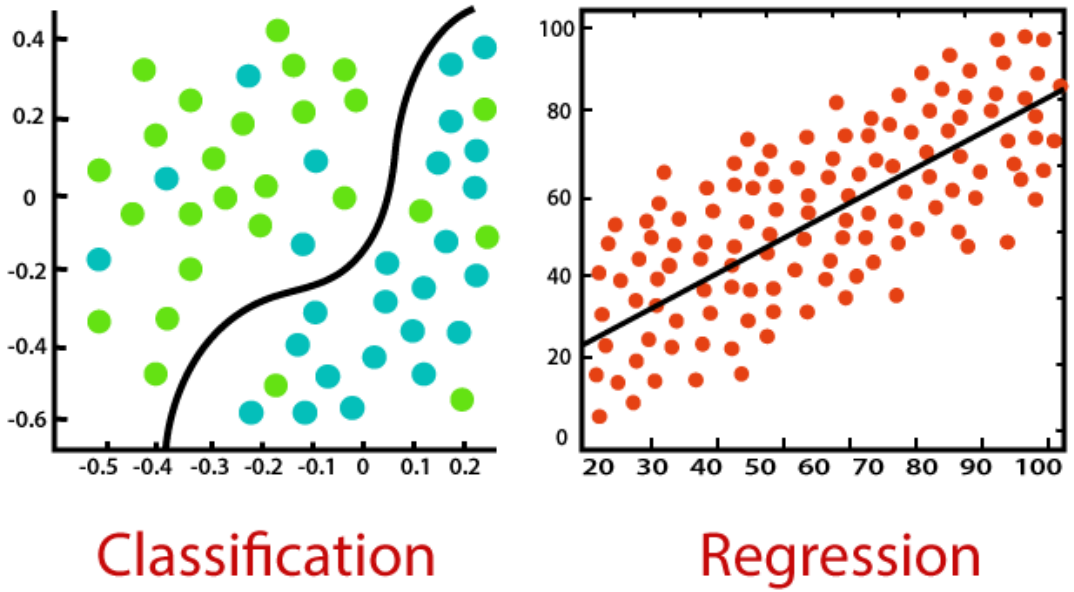


- Unsupervised Learning:
  - Clustering, Anomaly Detection, PCA, etc.
  - Find patterns in input data

- Unsupervised learning example:
  - Goal: Clustering, Outlier detection
  - Data: $[\overrightarrow{x_i}, \ldots, \overrightarrow{x_n}]$

Image Credits: wikipedia.org

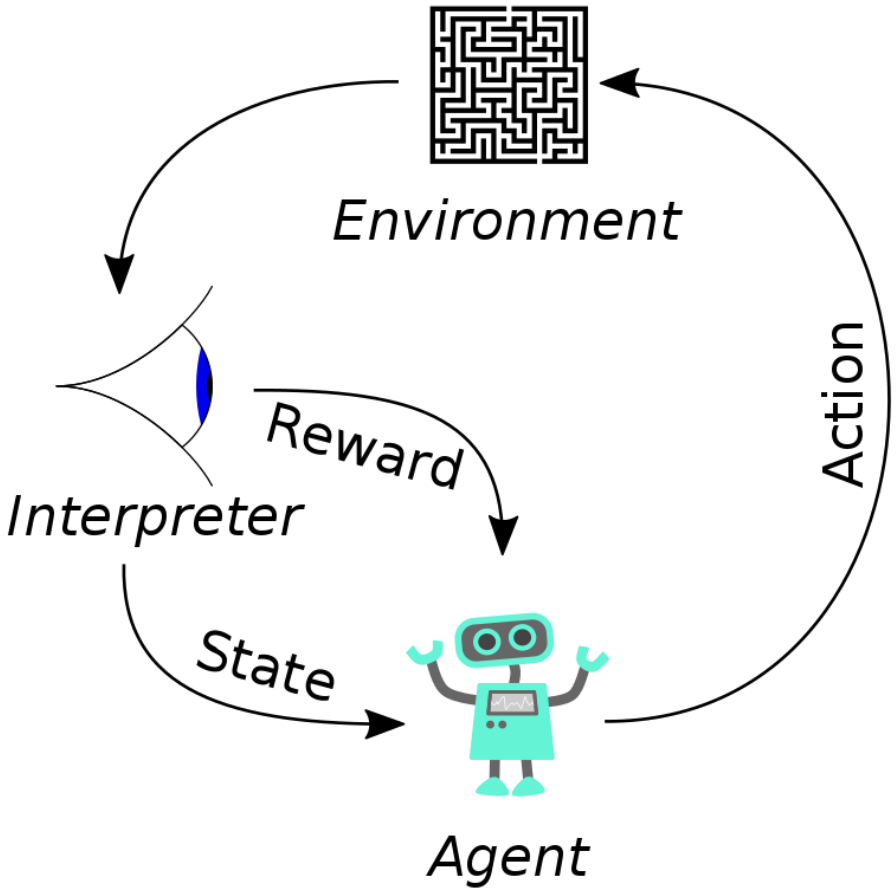
Machine Learning (ML)



- Supervised Learning:
  - Classification and Regression
  - Labelled data for training
- Regression:
  - Goal: $f(\vec{x}) = \vec{y}$,
  - Data: $[(\overrightarrow{x_i}, \overrightarrow{y_i}), \ldots, (\overrightarrow{x_n}, \overrightarrow{y_n})]$
- Classification:
  - Goal: $\text{argmax}\, P(class|\vec{x}) = \text{C}$
  - Data: $[(\overrightarrow{x_i}, C_i), \ldots, (\overrightarrow{x_n}, C_n)]$

Image Credits: javatpoint.com

Machine Learning (ML)



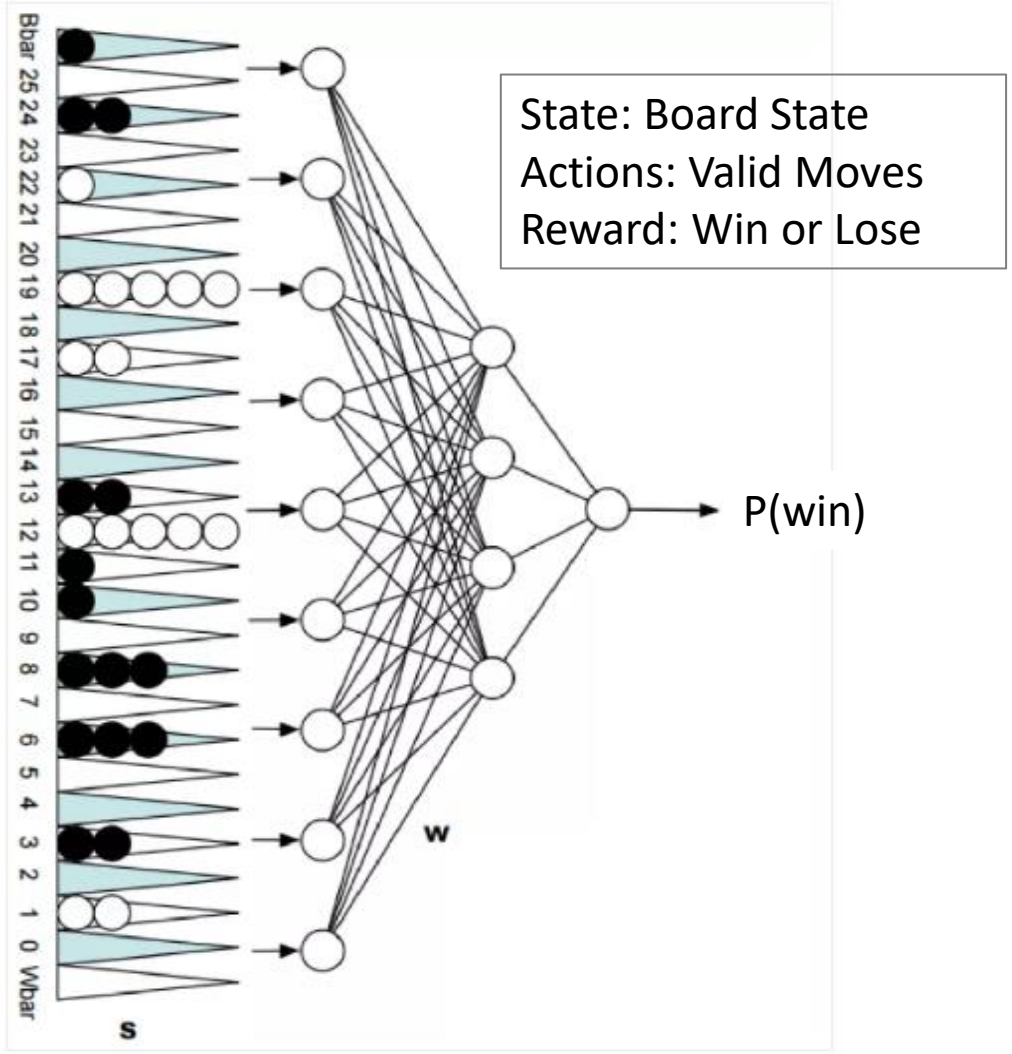
- Reinforcement Learning:
  - Stochastic optimal control
  - Learn to improve performance via interacting with the environment
- RL example:
  - Goal:
    - Maximize cumulative reward

      Maximize $\sum_{i=1}^{\infty} Reward(State_i, Act_i)$
  - Data:

    $Reward_{i+1}, State_{i+1} = Interact(State_i, Action_i)$

Image Credits: wikipedia.org

TD-Gammon – Tesauro ~1995



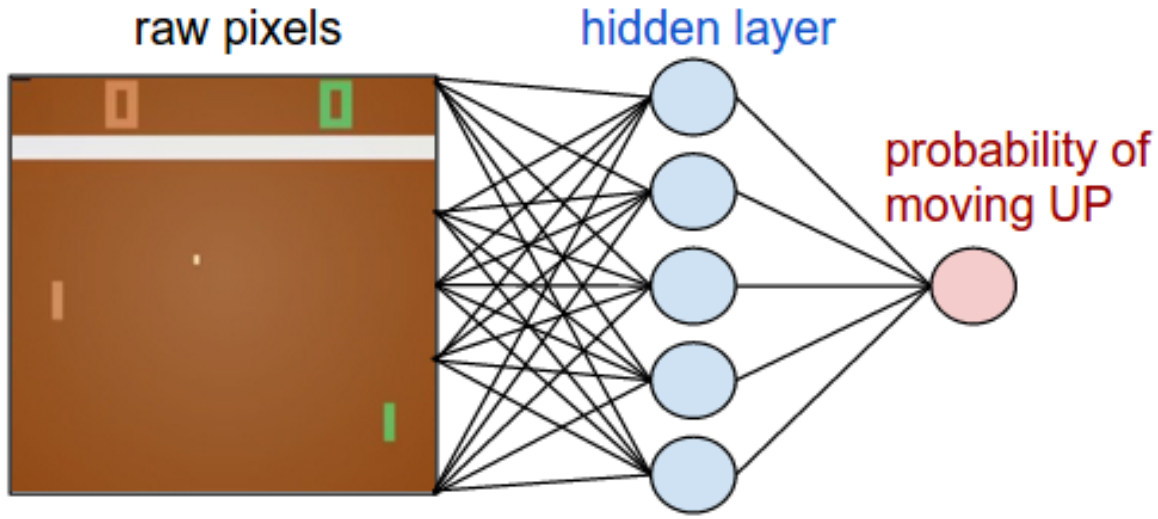
State: Board State
Actions: Valid Moves
Reward: Win or Lose

- Net with 80 hidden units, initialize to random weights
- Select move based on network estimate & shallow search
- Learn by playing against itself
- 1.5 million games of training
  - competitive with world class players

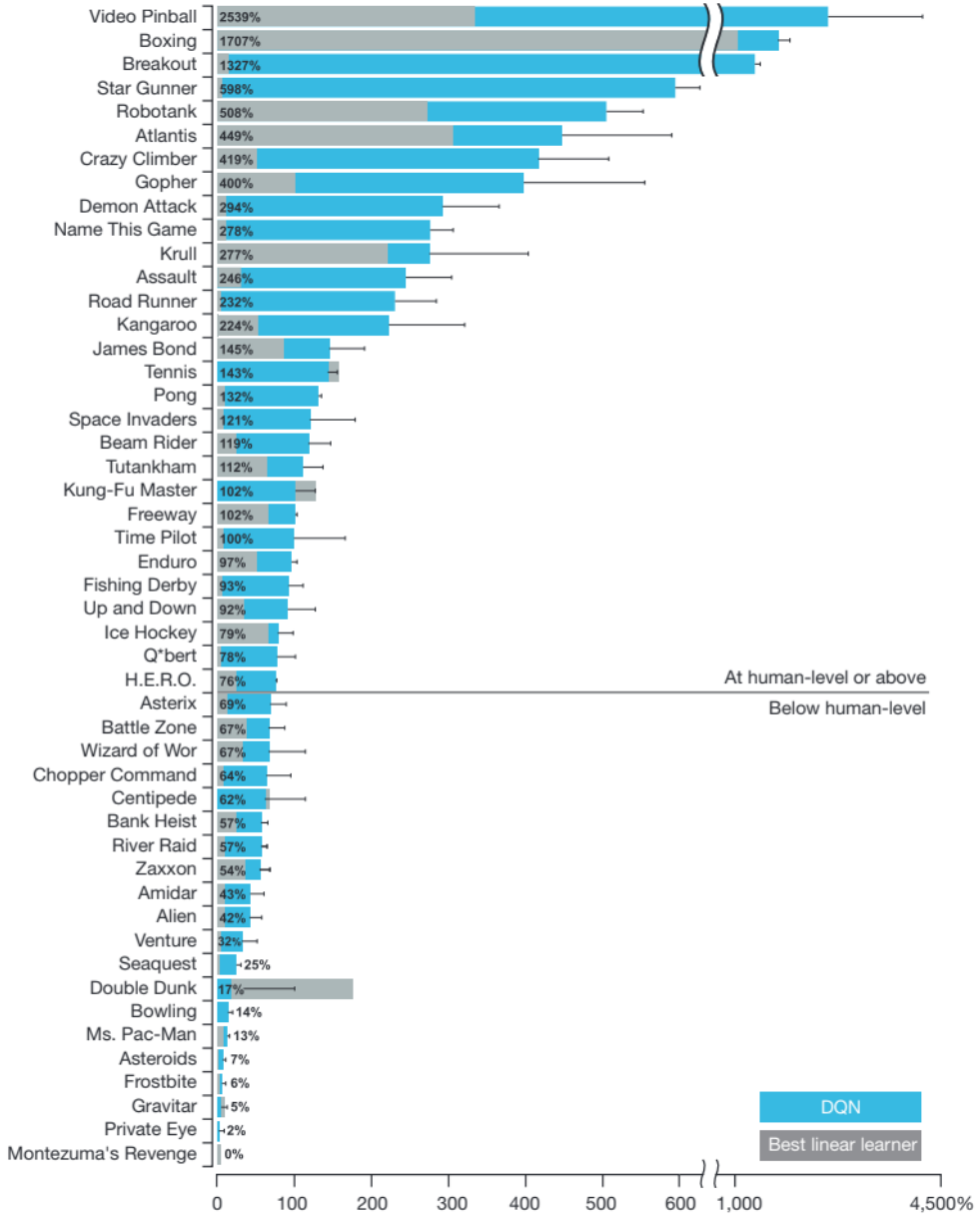
Slide Credits: Geoff Hulten

Atari 2600 games

State: Raw Pixels
Actions: Valid Moves
Reward: Game Score



Same model/parameters for ~50 games



Slide Credits: Geoff Hulten

Robotics and Locomotion

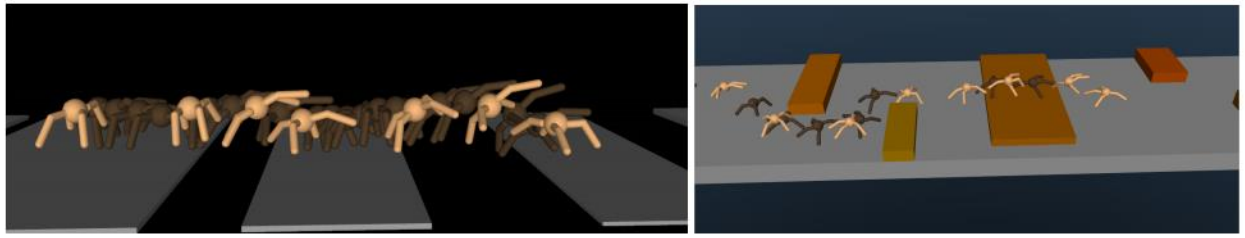
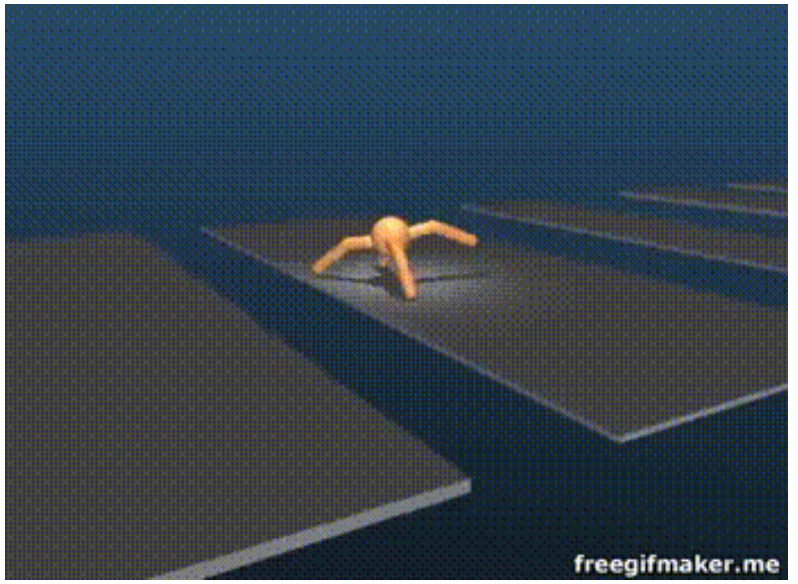


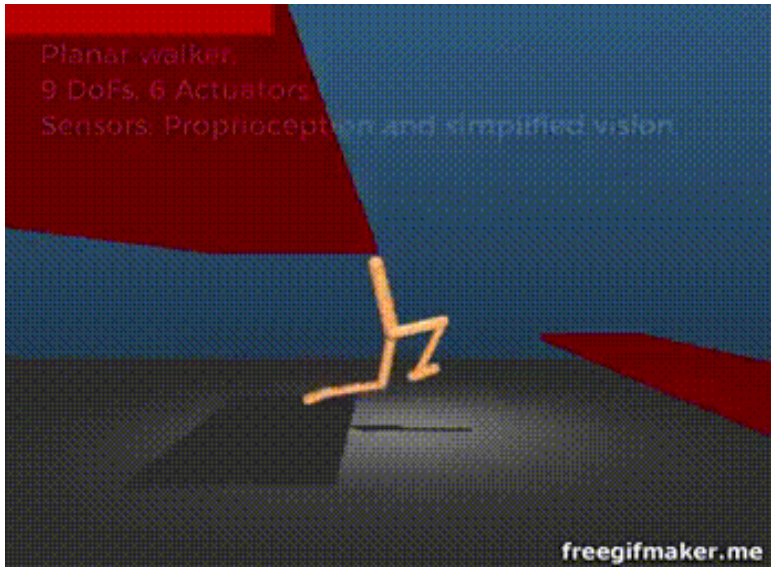
Figure 5: Time-lapse images of a representative *Quadruped* policy traversing gaps (left); and navigating obstacles (right)

State:
- Joint States/Velocities
- Accelerometer/Gyroscope
- Terrain

Actions: Apply Torque to Joints

Reward: Velocity – { stuff }





https://youtu.be/hx_bgoTF7bs

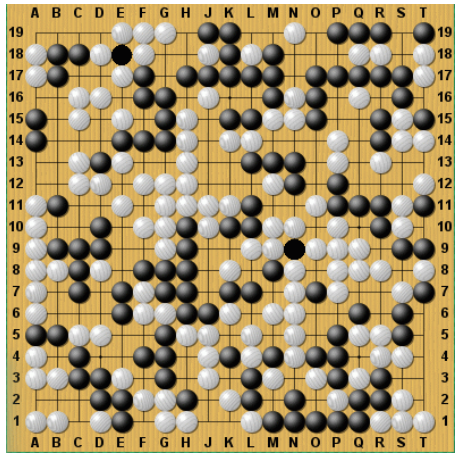
Slide Credits: Geoff Hulten

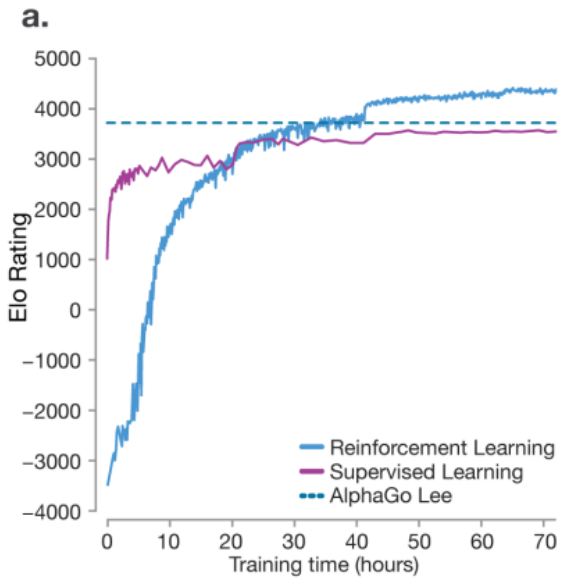
2017 paper https://arxiv.org/pdf/1707.02286.pdf

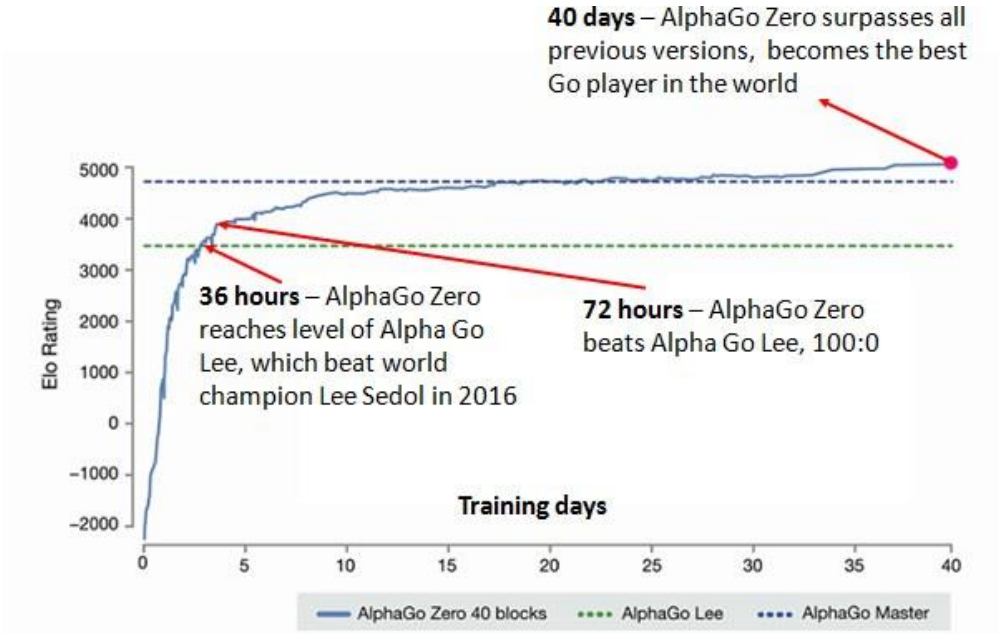
Alpha Go

State: Board State
Actions: Valid Moves
Reward: Win or Lose

- Learning how to beat humans at ‘hard’ games (search space too big)
- Far surpasses (Human) Supervised learning
- Algorithm learned to outplay humans at chess in 24 hours

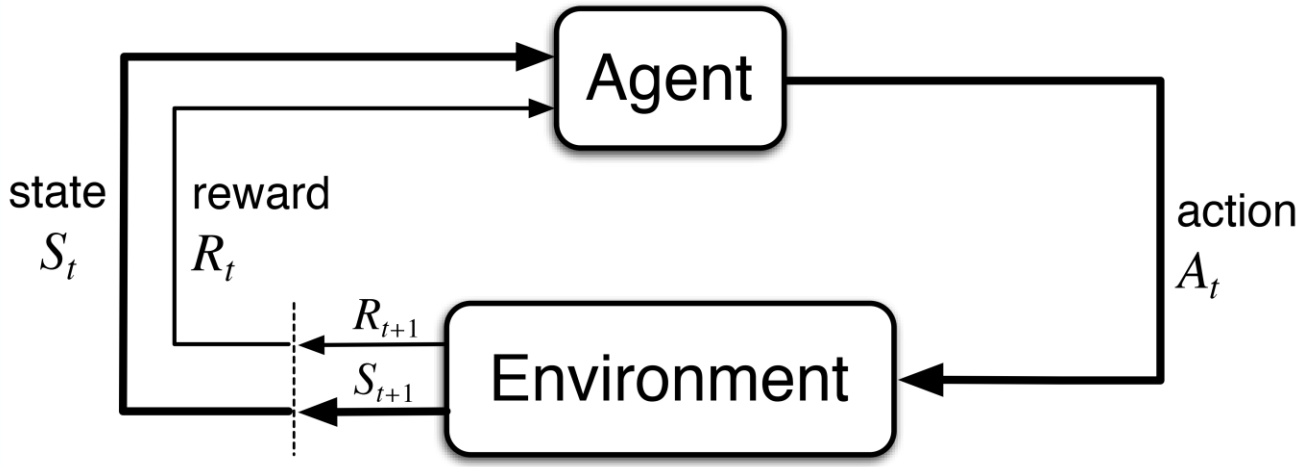






Slide Credits: Geoff Hulten

https://deepmind.com/documents/119/agz_unformatted_nature.pdf



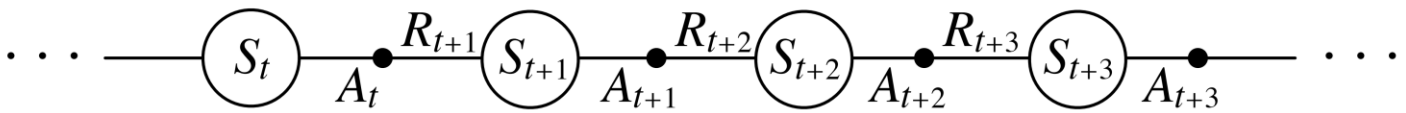
Agent and environment interact at discrete time steps: $t = 0, 1, 2, 3, \ldots$

Agent observes state at step t: $S_t \in \mathcal{S}$

produces action at step t : $A_t \in \mathcal{A}(S_t)$

gets resulting reward: $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$

and resulting next state: $S_{t+1} \in \mathcal{S}^+$

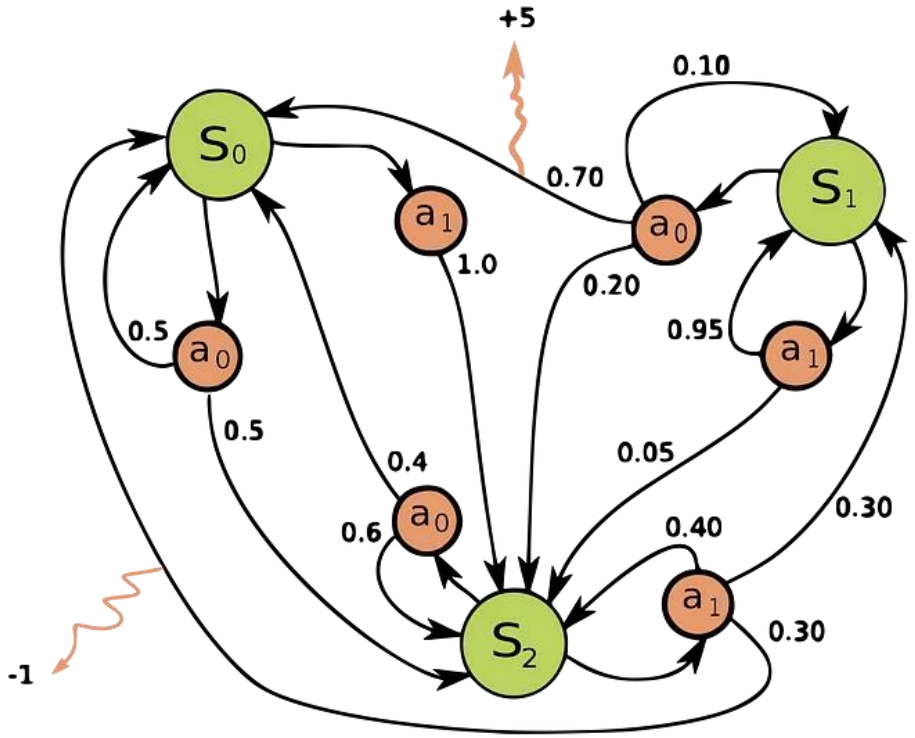


Slide Credits: Katerina Fragkiadaki

Reinforcement Learning

- RL problems are generally posed as Markov Decision Process (MDP)
  - RL is used for MDPs where the transition prob. or reward prob. are unknown.
- MDP: Discrete-Time Stochastic Control Process
  - Markovian Property:
    - Next reward and state does not depend on history.
    - Next reward and state depend only on current state and action.
  - It's a 4-tuple $\left(S, A_s, P_a(s, s'), R_a(s, s')\right)$
    - S → State Space → Set of states
    - A_s → Action Space available at state s → Set of possible actions
    - $P_a(s, s') = \mathbb{P}(s'|s, a)$ → Probability of transitioning from s to s′ after taking action a
    - $R_a(s, s') = \mathbb{E}(r'|s, a)$→ Expected Reward obtained after transitioning from s to s′ after taking action a

Example MDP



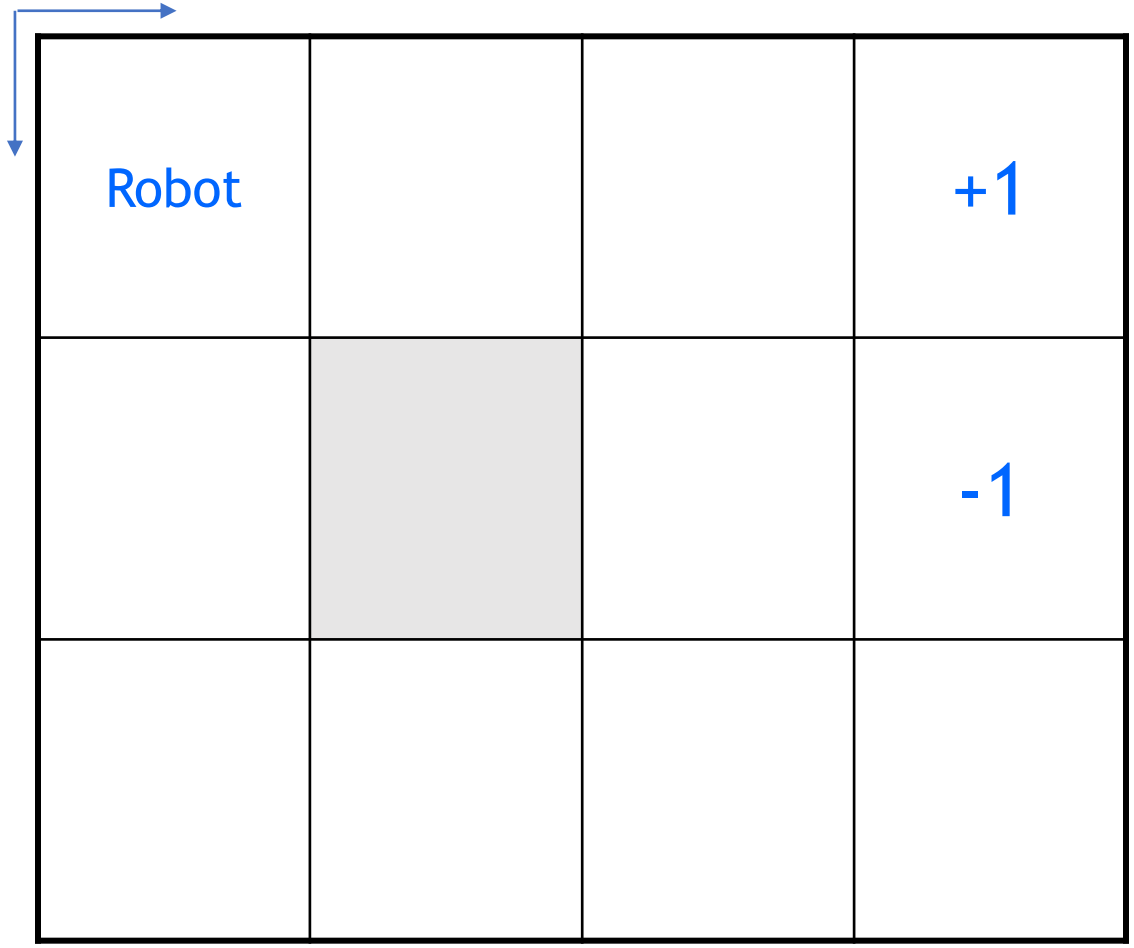
- 3 states (green circles)
  - $\{S_0, S_1, S_2\}$
- 2 actions (orange circles)
  - $\{a_0, a_1\}$
- 2 rewards (orange arrows)
  - $R_{a1}(S_2, S_0) = -1$
  - $R_{a0}(S_1, S_0) = +5$
- Example transition probabilities:
  - $P_{a0}(S_0, S_2) = 0.5$
  - $P_{a0}(S_0, S_0) = 0.5$

Image Credits: wikipedia.org

Reinforcement Learning

- Policy (π): Mapping from state to action
  - Deterministic: $a_t = \pi(s_t)$, or,
  - Stochastic: $a_t = argmax_a\, \pi(a|s_t)$
- Objective of RL:
  - Find a policy that maximizes long term cumulative reward.
  - maximize $\sum_{t=0}^{T} \gamma^t R_{a_t}(s_t, s_{t+1})$, where, $\gamma \in [0,1]$ (Discount factor)
- How to make a decision?
  - Rank State or (State, Action) based on some value derived from experience
  - Value functions measure the goodness of a particular state or state/action pair for a given Policy

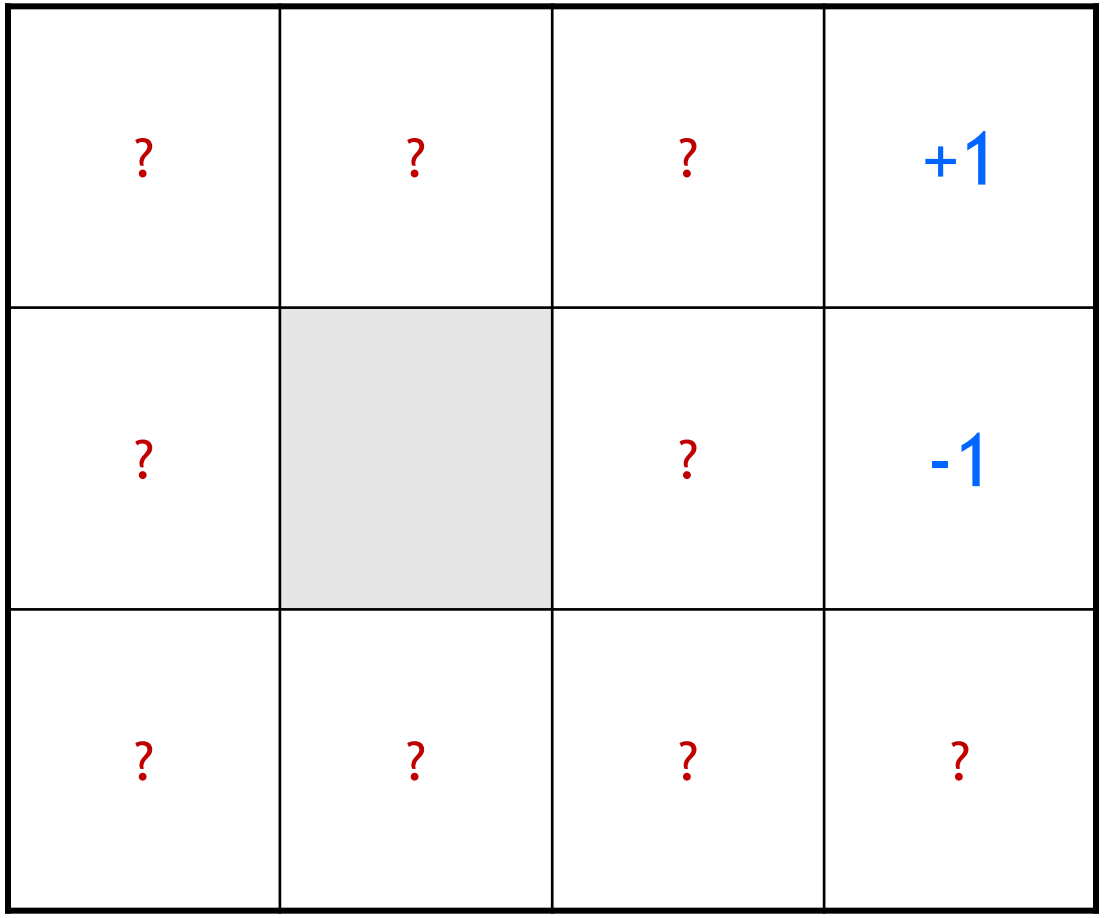
Robot in a room



- States:
  - Location ∈ $\{(1,1), \ldots, (3,4)\}$
- Actions:
  - UP, DOWN, LEFT, RIGHT
- Terminate at (1,4) or (2,4)
- Note:
  - Transitions and rewards are deterministic.
- Reward +1 at (1,4), -1 at (2,4)
- Reward -0.1 for each step

Slide based on works of **Peter Bodík,** RAD Lab, UC Berkeley

Robot in a room: State Value Function



Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V(s') \right) \right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$

Robot in a room: State Value Function

0	0	0	+1
0		0	-1
0	0	0	0

Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V(s') \right) \right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Value Iteration
  - Initializing
  - $V_{k+1}(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V_k(s') \right) \right)$

Robot in a room: State Value Function

-0.1	-0.1	0.9	+1
-0.1		-0.1	-1
-0.1	-0.1	-0.1	-0.1

Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V(s')\right)\right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Using Bellman equation iteratively
  - $V_{k+1}(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V_k(s')\right)\right)$
  - First Iteration

Robot in a room: State Value Function

-0.1	0.8	0.9	+1
-0.1		0.8	-1
-0.1	-0.1	-0.1	-0.1

Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s,s') + \gamma V(s')\right)\right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Using Bellman equation iteratively
  - $V_{k+1}(s) = \max_a \left(\mathbb{E}\left(R_a(s,s') + \gamma V_k(s')\right)\right)$
  - Second Iteration

Robot in a room: State Value Function

0.7	0.8	0.9	+1
-0.1		0.8	-1
-0.1	-0.1	0.7	-0.1

Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V(s')\right)\right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Using Bellman equation iteratively
  - $V_{k+1}(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V_k(s')\right)\right)$
  - Third Iteration

Robot in a room: State Value Function

0.7	0.8	0.9	+1
0.6		0.8	-1
-0.1	0.6	0.7	0.6

Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s,s') + \gamma V(s')\right)\right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Using Bellman equation iteratively
  - $V_{k+1}(s) = \max_a \left(\mathbb{E}\left(R_a(s,s') + \gamma V_k(s')\right)\right)$
  - Fourth Iteration

Robot in a room: State Value Function

0.7	0.8	0.9	+1
0.6		0.8	-1
0.5	0.6	0.7	0.6

Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V(s')\right)\right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Using Bellman equation iteratively
  - $V_{k+1}(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V_k(s')\right)\right)$
  - Fifth Iteration

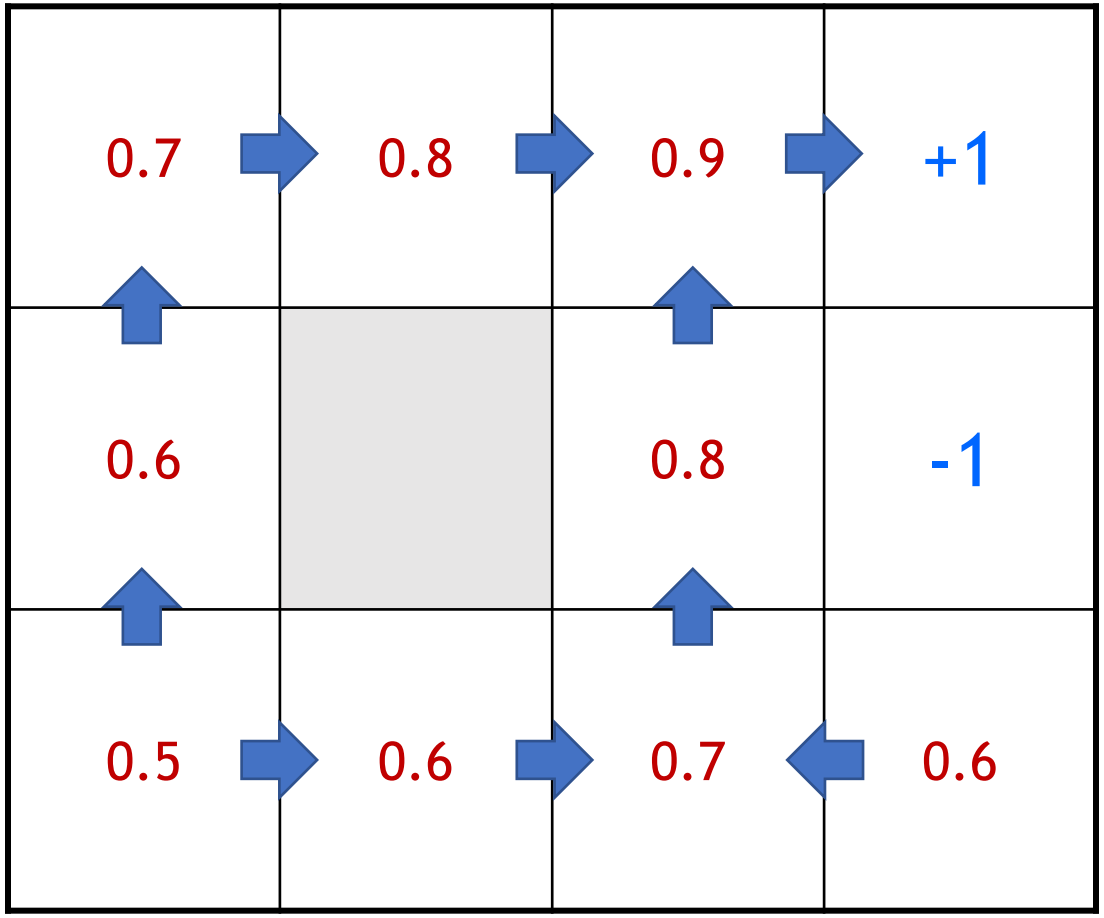
Robot in a room: State Value Function

0.7	0.8	0.9	+1
0.6		0.8	-1
0.5	0.6	0.7	0.6

Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s,s') + \gamma V(s')\right)\right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Using Bellman equation iteratively
  - $V_{k+1}(s) = \max_a \left(\mathbb{E}\left(R_a(s,s') + \gamma V_k(s')\right)\right)$
  - Converged

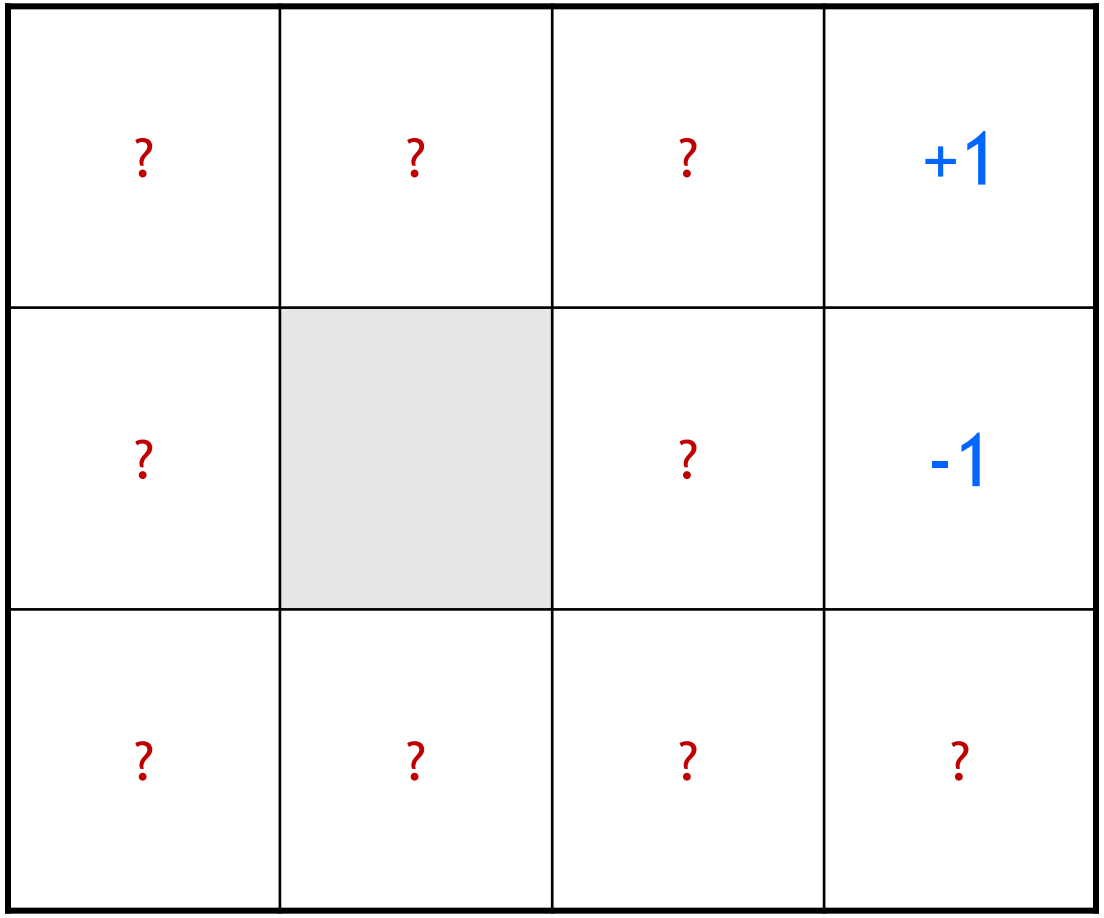
Robot in a room: State Value Function



Reward -0.1 for each step

- State Value Function:
  - $V(s)$
  - Maximum expected reward accumulated when starting from a given state
- Bellman equation (Optimal):
  - $V(s) = \max_a \left(\mathbb{E}\left(R_a(s, s') + \gamma V(s') \right) \right)$
  - $\gamma = 1, s = s_t, s' = s_{t+1}$
- Policy:
  - $\pi(s) = \operatorname{argmax}_a \sum P_a(s, s') V(s')$

Robot in a room:

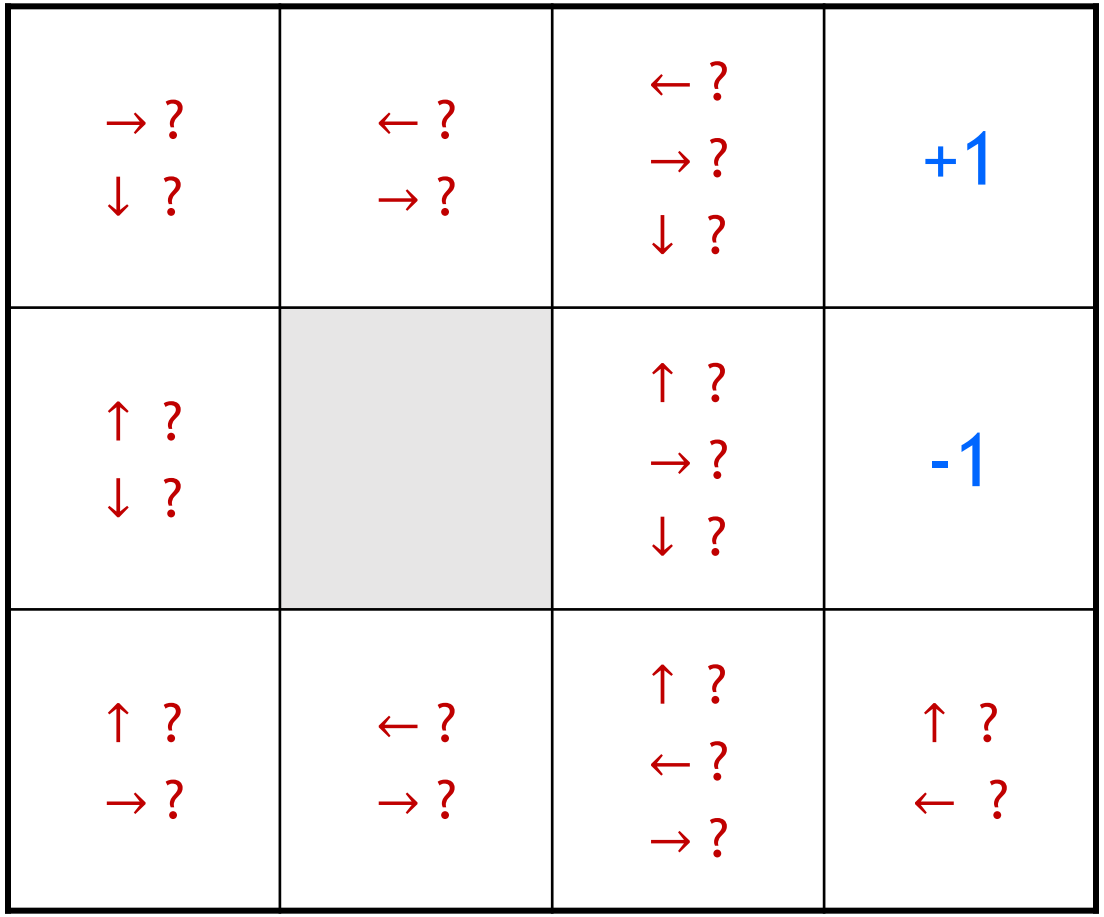


Reward -0.1 for each step

What if the robot is not functioning properly?

- State transitions are stochastic
  - An action may not lead to intended state
- Rewards/Costs are stochastic

Robot in a room: State-Action Value Function



Reward -0.1 for each step

- State-Action Value Function:
  - $Q(s,a)$
  - Maximum expected reward accumulated when starting from a given state and choosing a given action.
  - $V(s) = \max_a Q(s,a)$
- Bellman equation (Optimal):

$$Q(s,a) = \mathbb{E}\left(R_a(s,s') + \gamma \max_{a'} Q(s',a')\right)$$

$$\gamma = 1\ ,\ s = s_t,\ s' = s_{t+1},\ a' = a_{t+1}$$

Robot in a room: State-Action Value Function

→ 0 ↓ 0	← 0 → 0	← 0 → 0 ↓ 0	+1
↑ 0 ↓ 0		↑ 0 → 0 ↓ 0	-1
↑ 0 → 0	← 0 → 0	↑ 0 ← 0 → 0	↑ 0 ← 0

Reward -0.1 for each step

- State-Action Value Function:
  - $Q(s,a)$
  - Maximum expected reward accumulated when starting from a given state and choosing a given action.
  - $V(s) = \max_a Q(s,a)$
- Bellman equation (Optimal):

$$Q(s,a) = \mathbb{E}\left(R_a(s,s') + \gamma \max_{a'} Q(s',a')\right)$$

$$\gamma = 1\ ,\ s = s_t,\ s' = s_{t+1},\ a' = a_{t+1}$$

- Value Iteration: Initialization

Robot in a room: State-Action Value Function

→ -0.1 ↓ -0.1	← -0.1 → -0.1	← -0.1 → 0.9 ↓ -0.1	+1
↑ -0.1 ↓ -0.1		↑ -0.1 → -1.1 ↓ -0.1	-1
↑ -0.1 → -0.1	← -0.1 → -0.1	↑ -0.1 ← -0.1 → -0.1	↑ -1.1 ← -0.1

Reward -0.1 for each step

- State-Action Value Function:
  - $Q(s,a)$
  - Maximum expected reward accumulated when starting from a given state and choosing a given action.
  - $V(s) = \max_a Q(s,a)$
- Bellman equation (Optimal):

$$Q(s,a) = \mathbb{E}\left(R_a(s,s') + \gamma \max_{a'} Q(s',a')\right)$$

$$\gamma = 1\ ,\ s = s_t,\ s' = s_{t+1},\ a' = a_{t+1}$$

- Value Iteration: First Iteration

Robot in a room: State-Action Value Function

→ -0.2 ↓ -0.2	← -0.2 → 0.8	← -0.2 → 0.9 ↓ -0.2	+1
↑ -0.2 ↓ -0.2		↑ 0.8 → -1.1 ↓ -0.2	-1
↑ -0.2 → -0.2	← -0.2 → -0.2	↑ -0.2 ← -0.2 → -0.2	↑ -1.1 ← -0.2

Reward -0.1 for each step

- State-Action Value Function:
  - $Q(s,a)$
  - Maximum expected reward accumulated when starting from a given state and choosing a given action.
  - $V(s) = \max_a Q(s,a)$
- Bellman equation (Optimal):

$$Q(s,a) = \mathbb{E}\left(R_a(s,s') + \gamma \max_{a'} Q(s',a')\right)$$

$$\gamma = 1\ ,\ s = s_t,\ s' = s_{t+1},\ a' = a_{t+1}$$

- Value Iteration: Second Iteration

Robot in a room: State-Action Value Function

→ 0.7 ↓ -0.3	← -0.3 → 0.8	← 0.7 → 0.9 ↓ 0.7	+1
↑ -0.3 ↓ -0.3		↑ 0.8 → -1.1 ↓ -0.3	-1
↑ -0.3 → -0.3	← -0.3 → -0.3	↑ 0.7 ← -0.3 → -0.3	↑ -1.1 ← -0.3

Reward -0.1 for each step

- State-Action Value Function:
  - $Q(s,a)$
  - Maximum expected reward accumulated when starting from a given state and choosing a given action.
  - $V(s) = \max_a Q(s,a)$
- Bellman equation (Optimal):

$$Q(s,a) = \mathbb{E}\left(R_a(s,s') + \gamma \max_{a'} Q(s',a')\right)$$

$$\gamma = 1\ ,\ s = s_t,\ s' = s_{t+1},\ a' = a_{t+1}$$

- Value Iteration: Third Iteration

Robot in a room: State-Action Value Function

→ 0.7 ↓ 0.5	← 0.6 → 0.8	← 0.7 → 0.9 ↓ 0.7	+1
↑ 0.6 ↓ 0.4		↑ 0.8 → -1.1 ↓ 0.6	-1
↑ 0.5 → 0.5	← 0.4 → 0.6	↑ 0.7 ← 0.5 → 0.5	↑ -1.1 ← 0.6

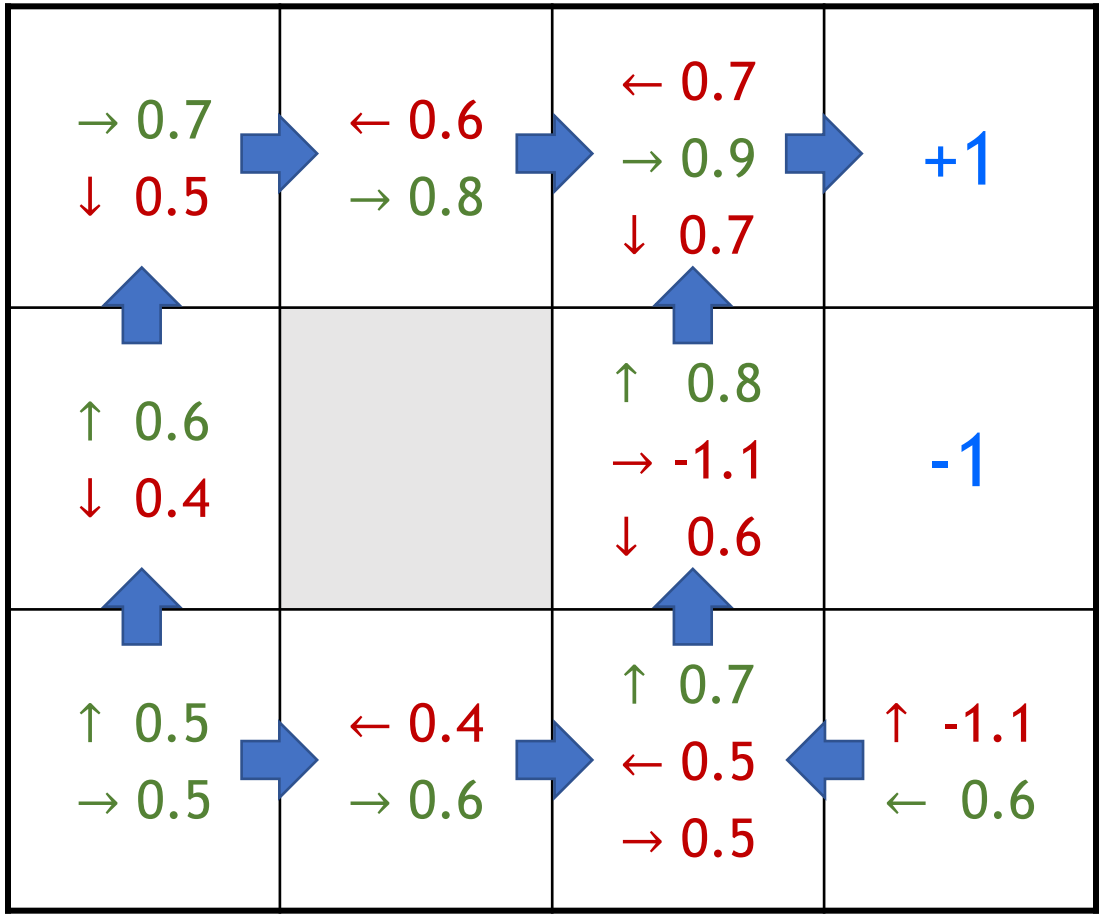
Reward -0.1 for each step

- State-Action Value Function:
  - $Q(s,a)$
  - Maximum expected reward accumulated when starting from a given state and choosing a given action.
  - $V(s) = \max_a Q(s,a)$
- Bellman equation (Optimal):

$$Q(s,a) = \mathbb{E}\left(R_a(s,s') + \gamma \max_{a'} Q(s',a')\right)$$

$$\gamma = 1\ ,\ s = s_t,\ s' = s_{t+1},\ a' = a_{t+1}$$

Robot in a room: State-Action Value Function

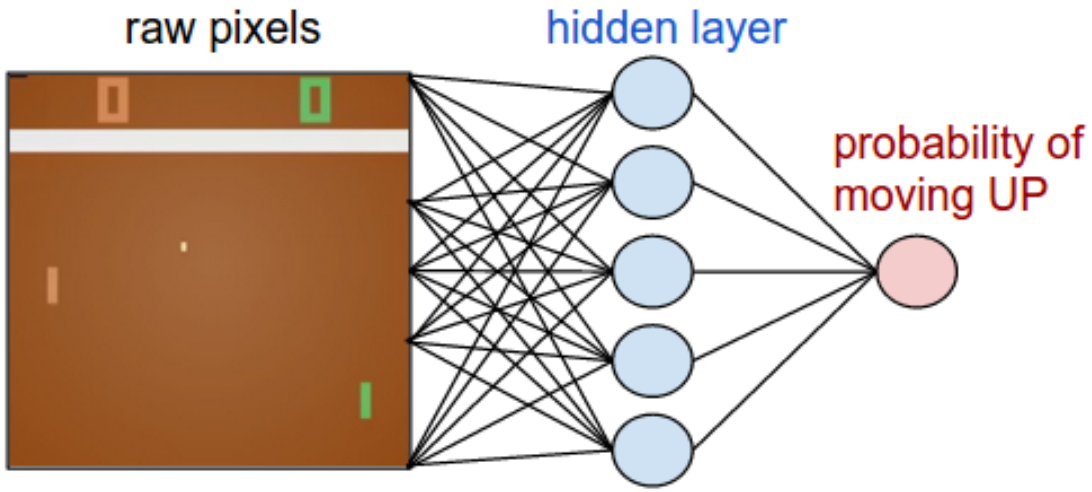


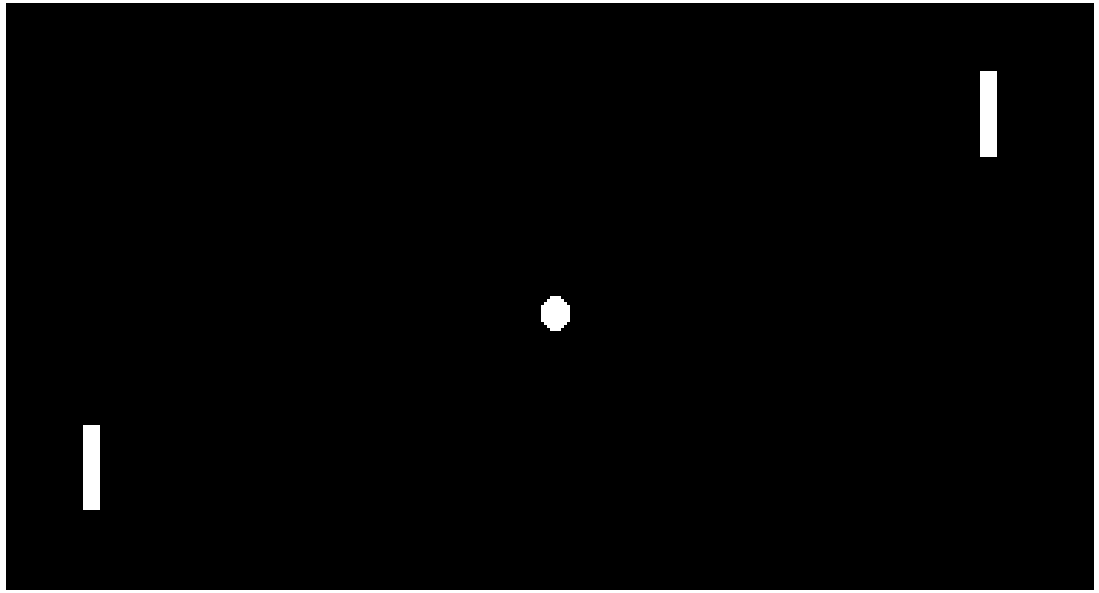
Reward -0.1 for each step

- State-Action Value Function:
  - $Q(s, a)$
  - Maximum expected reward accumulated when starting from a given state and choosing a given action.
  - $V(s) = \max_a Q(s, a)$
- Policy:
  - $\pi(s) = argmax_a Q(s, a)$

Policy Gradient Method

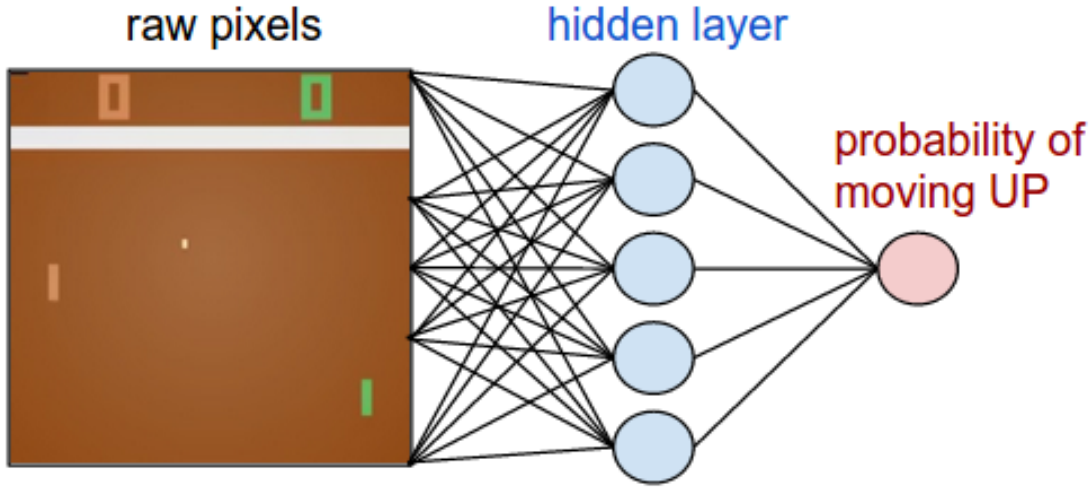
- Policy Gradient:
  - learn policy directly $\pi(a|s)$





Example from: https://www.youtube.com/watch?v=tqrcjHuNdmQ

Policy Gradient Method



- Policy Gradient:
  - learn policy directly $\pi(a|s)$

- REINFORCE Algorithm

Initialize θ arbitrarily

for each episode $\{s_0, a_0, r_0, \ldots, s_T, a_T, r_T\} \sim \pi(\cdot, \cdot; \theta)$

$$\textbf{do} \begin{cases} \textbf{for } t \leftarrow 0 \textbf{ to } T \\ \quad \textbf{do} \begin{cases} G \leftarrow \sum_{k=t}^{T} \gamma^{k-t} \cdot r_k \\ \theta \leftarrow \theta + \alpha \cdot \gamma^t \cdot \nabla_\theta \log \pi(s_t, a_t; \theta) \cdot G \end{cases} \end{cases}$$

return (θ)

References

- “What is reinforcement learning?,” What Is Reinforcement Learning? - MATLAB & Simulink, https://www.mathworks.com/discovery/reinforcement-learning.html (accessed Jul. 10, 2023).
- S. Brown, “Machine Learning, explained,” MIT Sloan, https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained (accessed Jul. 10, 2023).
- “Cluster analysis,” Wikipedia, https://en.wikipedia.org/wiki/Cluster_analysis (accessed Jul. 10, 2023).
- “Regression vs classification in Machine Learning - Javatpoint,” www.javatpoint.com, https://www.javatpoint.com/regression-vs-classification-in-machine-learning (accessed Jul. 10, 2023).
- “Reinforcement learning,” Wikipedia, https://en.wikipedia.org/wiki/Reinforcement_learning (accessed Jul. 10, 2023).
- G. Hulten, “CSEP546: Machine learning,” University of Washington, https://courses.cs.washington.edu/courses/csep546/18au/ (accessed Jul. 10, 2023).

References

- K. Fragkiadaki and T. Mitchell, “CMU 10703: Deep RL and Control,” Carnegie Mellon University, https://www.andrew.cmu.edu/course/10-703/ (accessed Jul. 10, 2023).
- “Markov decision process,” Wikipedia, https://en.wikipedia.org/wiki/Markov_decision_process (accessed Jul. 10, 2023).
- P. Bodik, “Practical machine learning lecture: Reinforcement learning,” University of California, Berkeley, http://people.eecs.berkeley.edu/~jordan/courses/294-fall09/lectures/reinforcement/ (accessed Jul. 10, 2023).
- A. Karpathy et al., “Deep RL Bootcamp,” Deep RL Bootcamp - Berkeley CA, https://sites.google.com/view/deep-rl-bootcamp/lectures (accessed Jul. 10, 2023).
- R. S. Sutton and A. Barto, Reinforcement Learning: An Introduction. Cambridge, Massachusetts ; London, England: The MIT Press, 2020.

Thankyou

Appendices

A1. Optimal State-Value Function

- Value function for arbitrary π

$$\begin{aligned} v_\pi(s) &\doteq \mathbb{E}_\pi[G_t|S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\Big[r + \gamma v_\pi(s')\Big] \end{aligned}$$

- Optimal value function

$$\begin{aligned} v_*(s) &\doteq \max_\pi v_\pi(s) \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1})|S_t = s, A_t = a] \\ &= \max_a \sum_{s',r} p(s', r|s, a)\Big[r + \gamma v_*(s')\Big] \end{aligned}$$

- Return

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... \\ &= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

A2. Optimal (State, Action)-Value Function

- Q function for arbitrary π

$$q_\pi(s,a) \doteq \mathbb{E}_\pi[G_t|S_t=s,A_t=a]$$
$$= \sum_{s',r} p(s',r|s,a)\Big[r+\gamma\sum_{a'}\pi(a'|s')q_\pi(a',s')\Big]$$

- Optimal Q function

$$q_*(s,a) \doteq \max_\pi q_\pi(s,a)$$
$$= \mathbb{E}[R_{t+1}+\gamma\max_{a'} q_*(S_{t+1},a')|S_t=s,A_t=a]$$
$$= \sum_{s',r} p(s',r|s,a)\Big[r+\gamma\max_{a'} q_*(s',a')\Big]$$

- Return

$$G_t \doteq R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\ldots$$
$$= \sum_{k=0}^{\infty}\gamma^k R_{t+k+1}$$
$$= R_{t+1}+\gamma G_{t+1}$$

A3. Value iteration

Params: θ - a small positive threshold determining the accuracy of the estimation

Initialize V(s), for all $s \in \mathcal{S}^+$ arbitrarily, except V(terminal)

$\Delta \leftarrow 0$

while $\Delta \geq \theta$ **do**

  foreach $s \in S$ **do**

    $v \leftarrow V(s)$

    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V(s')\right]$

    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

  end

end

output: Deterministic policy $\pi \approx \pi_*$ such that

$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V(s')\right]$

Reinforcement Learning

Reinforcement learning

- Reinforcement Learning is the agent must sense the environment, learns to behave (act) in a environment by performing actions (reinforcement) and seeing the results.
- Task
  - Learn how to behave successfully to achieve a goal while interacting with an external environment.
  - The goal of the agent is to **learn** an **action policy** that maximizes the total reward it will receive from any starting state.
- Examples
  - **Game playing:** player knows whether it win or lose, but not know how to move at each step

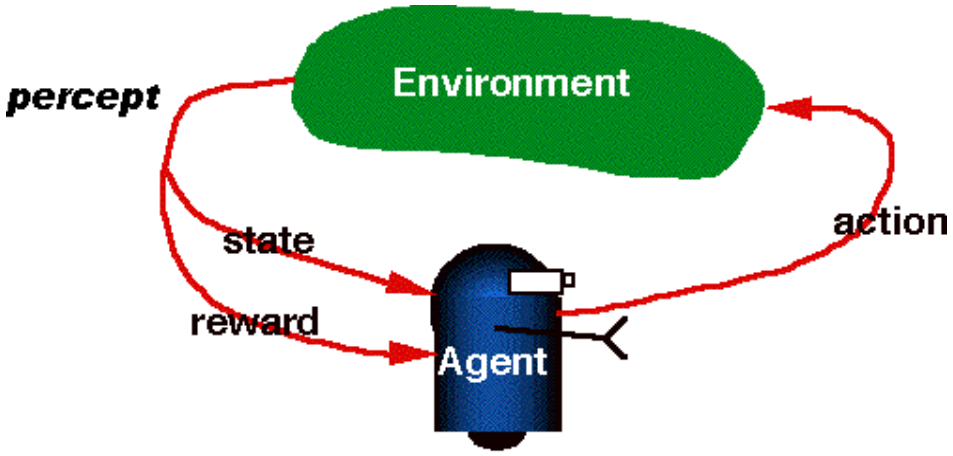
Applications

- A robot cleaning the room and recharging its battery
- Robot-soccer
- invest in shares
- Modeling the economy through rational agents
- Learning how to fly a helicopter
- Scheduling planes to their destinations

Reinforcement Learning Process

- RL contains two primary components:
  1. Agent (A) – RL algorithm that learns from trial and error
  2. Environment – World Space in which the agent moves (interact and take action)
- State (S) – Current situation returned by the environment
- Reward (R) – An immediate return from the environment to appraise the last action
- Policy (π) –Agent uses this approach to decide the next action based on the current state
- Value (V) – Expected long-term return with discount. Oppose to the short-term reward (R)
- Action-Value (Q) – Similar to value except it contains an additional parameter, the current action (A)

Figure shows RL is learning from interaction



RL Approaches

- Two approaches
  - *Model based approach RL*:
    - learn the model, and use it to derive the optimal policy.

      e.g Adaptive dynamic learning(ADP) approach
  - *Model free approach RL*:
    - derive the optimal policy without learning the model.

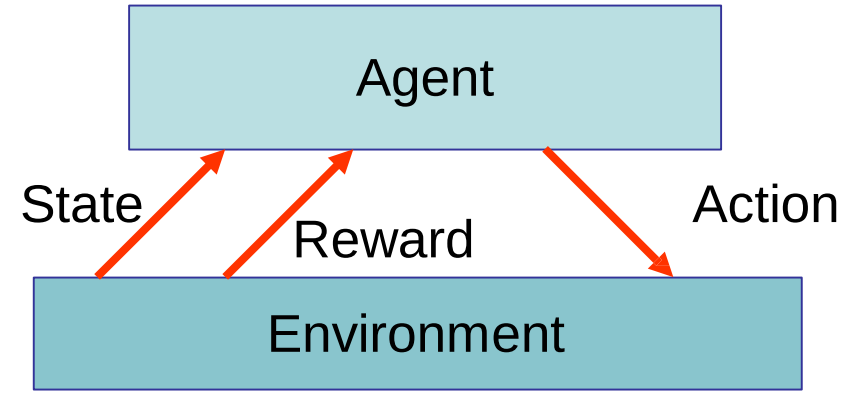
      e.g LMS and Temporal difference approach
- Passive learning
  - The agent imply watches the world during transition and tries to learn the utilities in various states
- Active learning
  - The agent not simply watches, but also acts on the environment

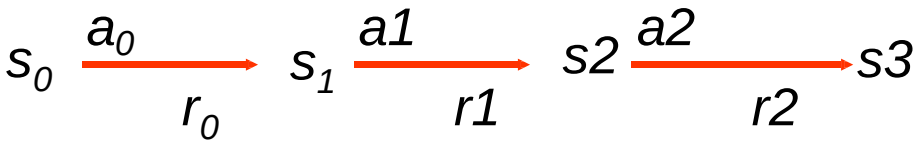
Reinforcement learning model

- Each percept(*e*) is enough to determine the State(the state is accessible)
- **Agent's task:** Find a optimal policy by mapping states of environment to actions of the agent, that maximize long-run measure of the reward (reinforcement)
- It can be modeled as Markov Decision Process (MDP) model.
  - Markov decision process (**MDP**) is a a mathematical framework for **modeling** decision making i.e mapping a solution in reinforcement learning.

MDP model

- MDP model <S,T,A,R>





- S– set of states
- A– set of actions
- **Transition Function:** T(s,a,s') = P(s'|s,a) – the probability of transition from *s* to *s'* given action *a*

  T(s,a) → s'
- **Reward Function:** r(s,a) → r the expected reward for taking action *a* in state *s*

$$R(s,a) = \sum_{s'} P(s' \mid s,a) r(s,a,s')$$

$$R(s,a) = \sum_{s'} T(s,a,s') r(s,a,s')$$

Q - Learning

- **Q-Learning** is a value-based **reinforcement learning** algorithm uses Q-values (action values) to iteratively improve the behavior of the learning agent.
- Goal is to maximize the Q value to find the optimal action-selection policy.
- The Q table helps to find the best action for each state and maximize the expected reward.
- **Q-Values / Action-Values:** Q-values are defined for states and actions.
- *Q(s, a)* denotes an estimation of the action *a* at the state *s.*
- This estimation of *Q(s, a)* will be iteratively computed using the **TD-Update rule.**
- **Reward:** At every transition, the agent observes a reward for every action from the environment, and then transits to another state.
- **Episode:** If at any point of time the agent ends up in one of the terminating states i.e. there are no further transition possible is called **completion of an episode.**

Q-Learning

- Initially agent explore the environment and update the Q-Table. When the Q-Table is ready, the agent will start to exploit the environment and taking better actions.
- It is an off-policy control algorithm i.e. the updated policy is different from the behavior policy. It estimates the reward for future actions and appends a value to the new state without any greedy policy. Temporal difference(TD) learning is one of the model-free reinforcement learning techniques

Temporal Difference or TD-Update:

- Estimate the value of Q is applied at every time step of the agents interaction with the environment. $Q(S,A) \leftarrow Q(S,A) + \alpha\left(R + \gamma Q(S',A') - Q(S,A)\right)$

Advantage:

- Converges to an optimal policy in both deterministic and nondeterministic MDPs.

Disadvantage:

- Suitable for small problems

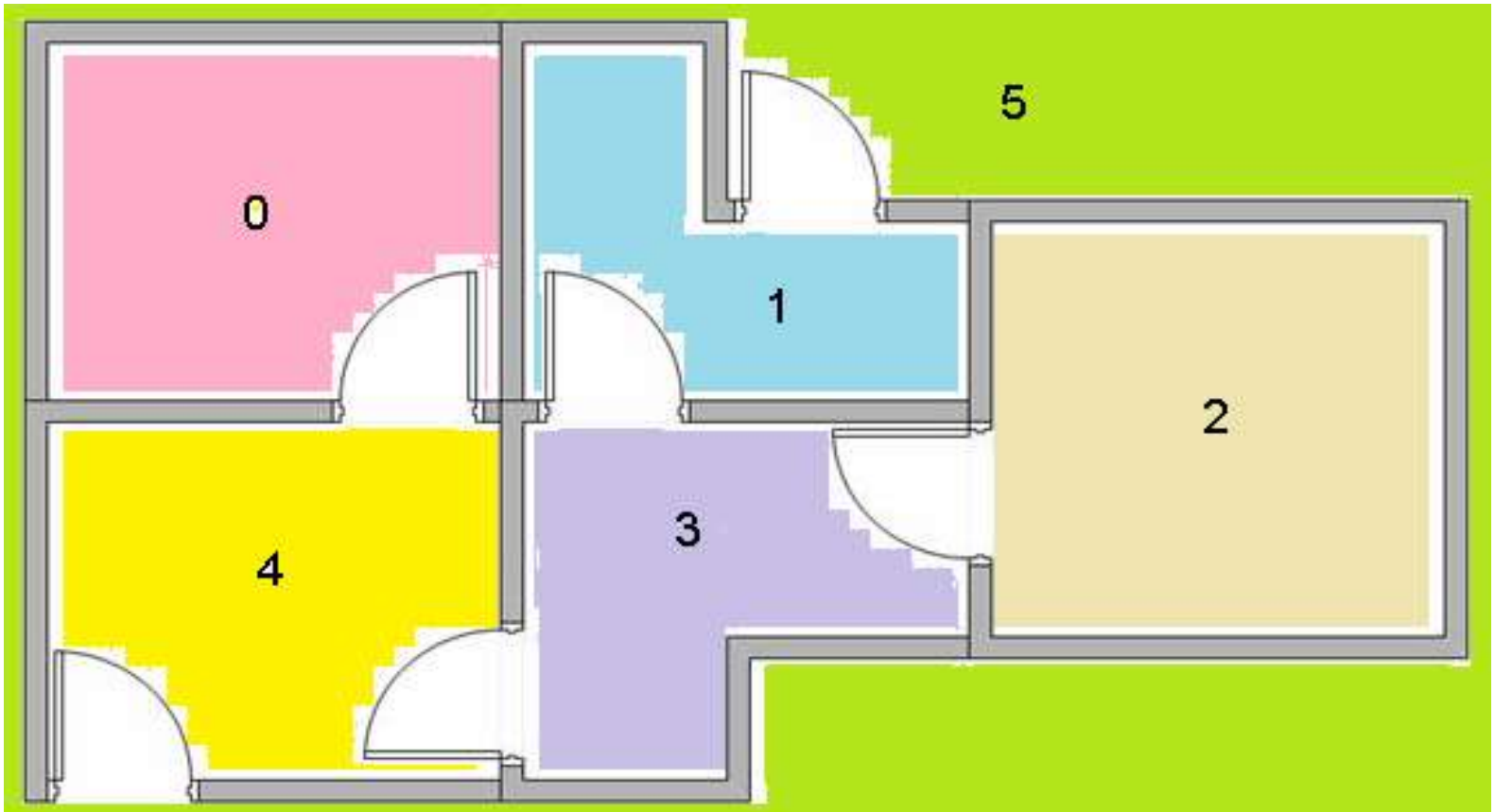
Bellman Equation

In the context of Q-learning, Bellman's equation is utilized to determine the value of a specific state and evaluate its relative placement.
The optimal state is determined by the state with the highest value.
The Bellman's equation can be represented as

Q(state, action) = R(state, action) + Gamma * max[Q(next state, all actions)]
Where,
Q(s,a) represents the expected reward for an action 'a' in state 's'.
R (s,a) represents the reward earned when action a is carried out in state 's'.
α is the discount factor, which denotes the significance of future rewards.
maxaQ(s',a) represents the maximum Q-value for the next state s' and every possible action.

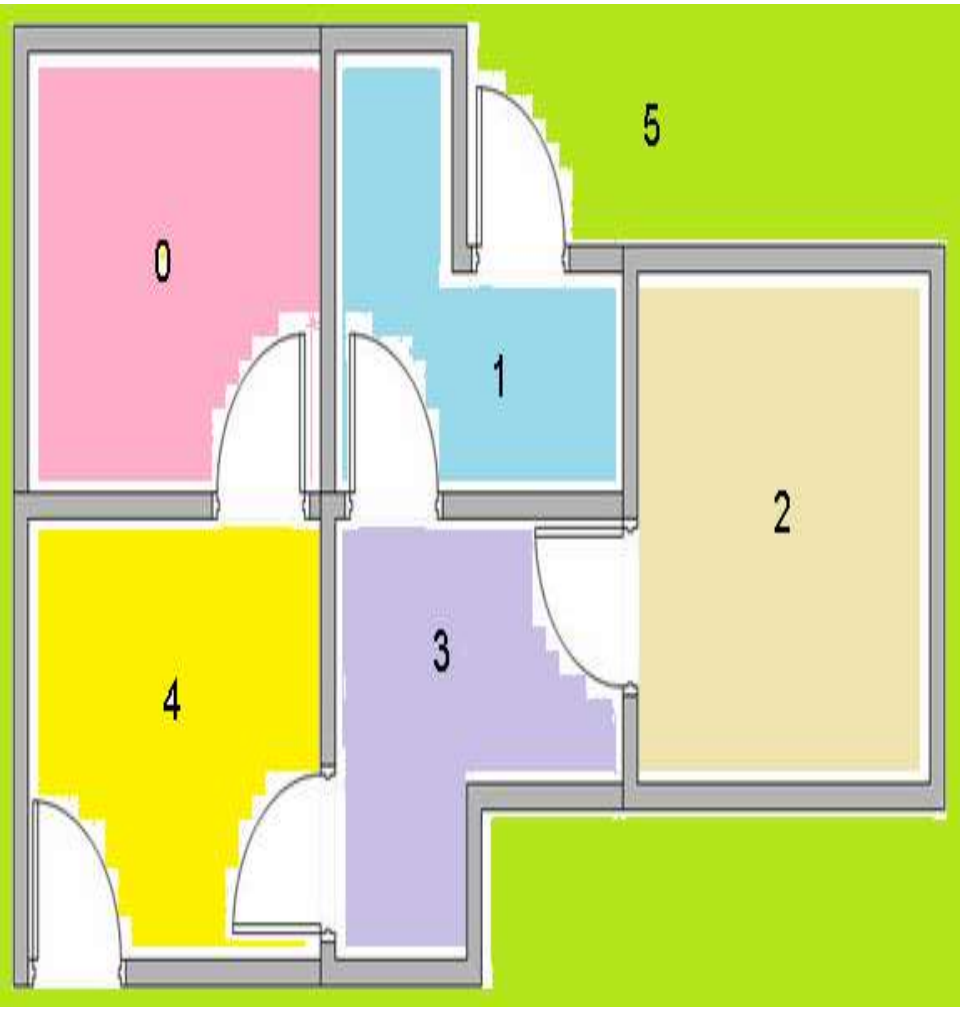
Understanding the Q – Learning

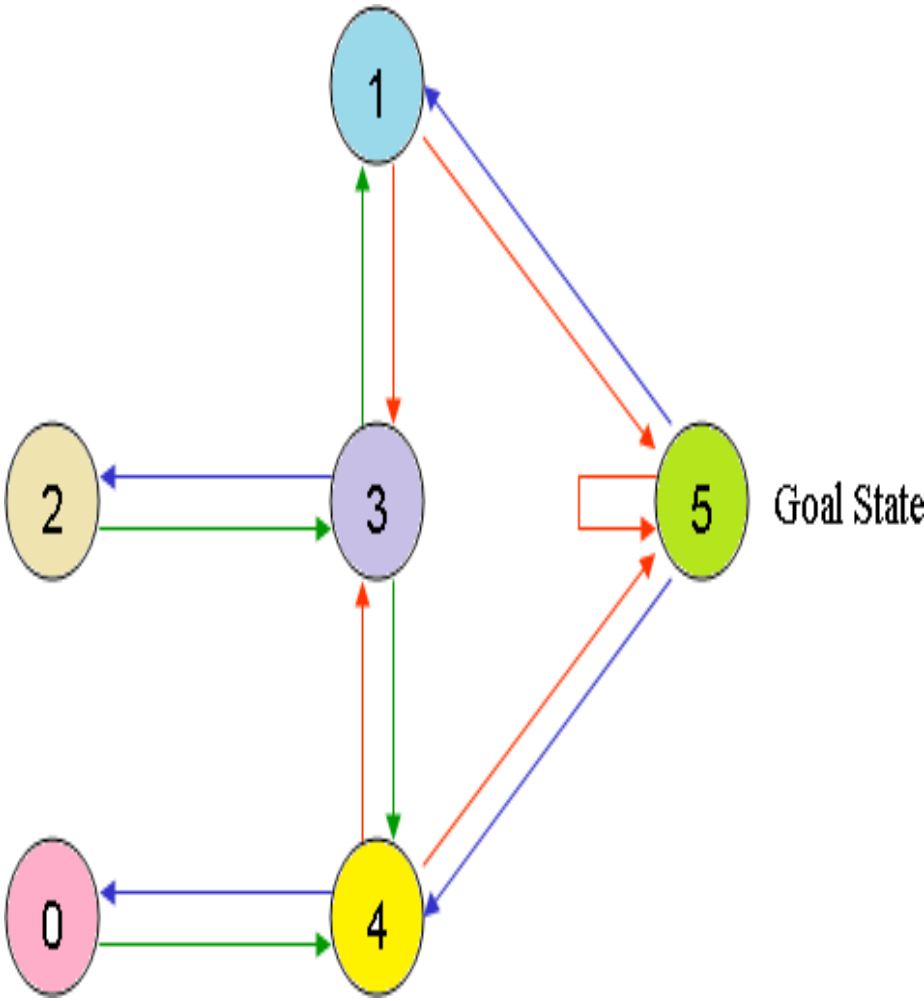
- Building Environment contains 5 rooms that are connected with doors.
- Each room is numbered from 0 to 4. The building outside is numbered as 5.
- Doors from room 1 and 4 leads to the building outside 5.
- **Problem:** Agent can place at any one of the rooms (0, 1, 2, 3, 4). Agent's goal is to reach the building outside (room 5).



Understanding the Q – Learning

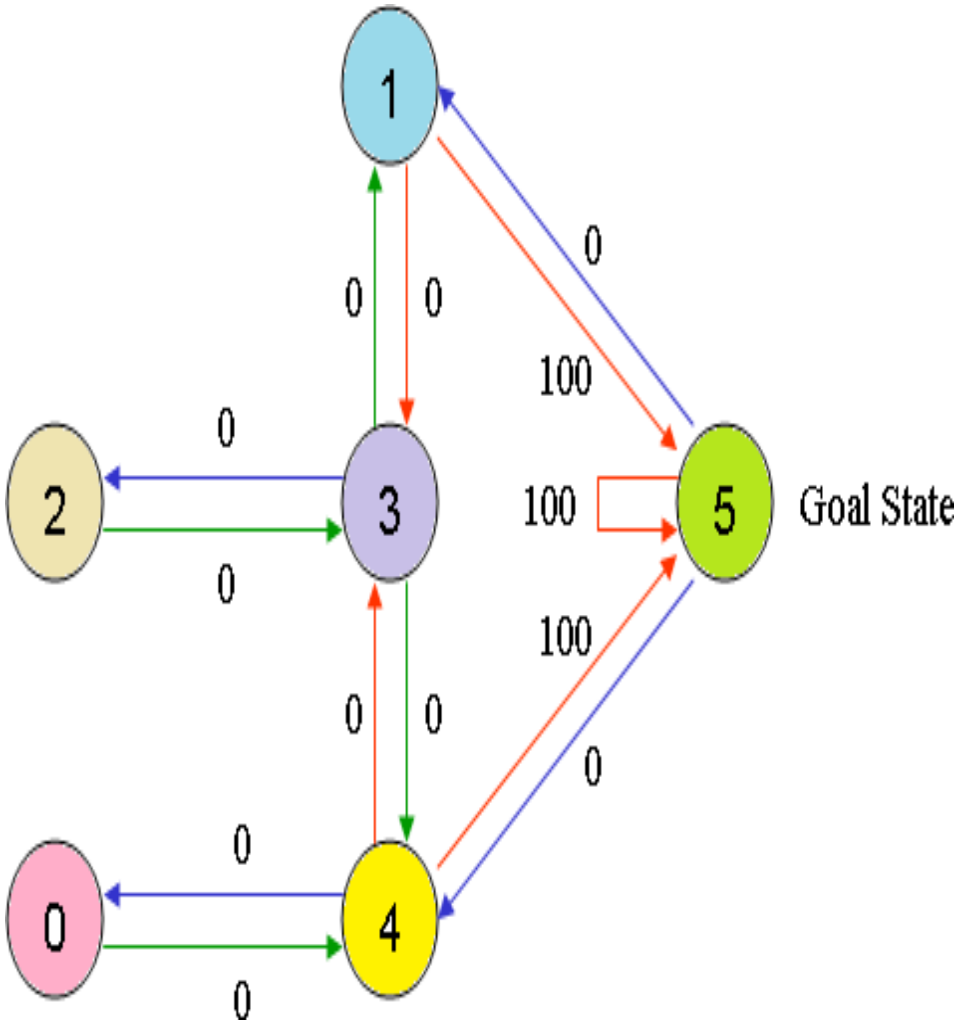
- Represent the room in the graph.
- Room number is the state and door is the edge.





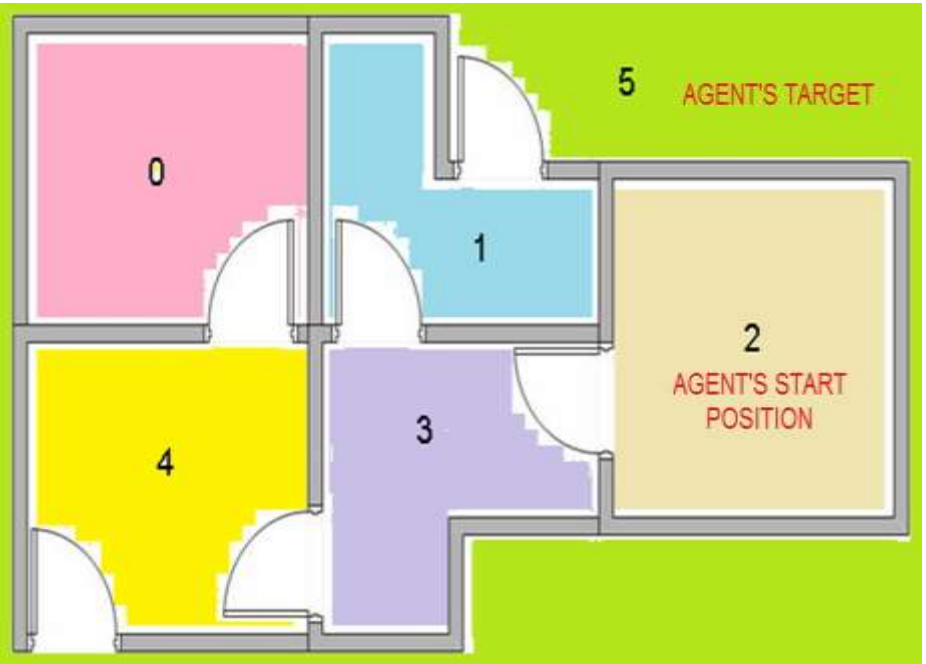
Understanding the Q – Learning

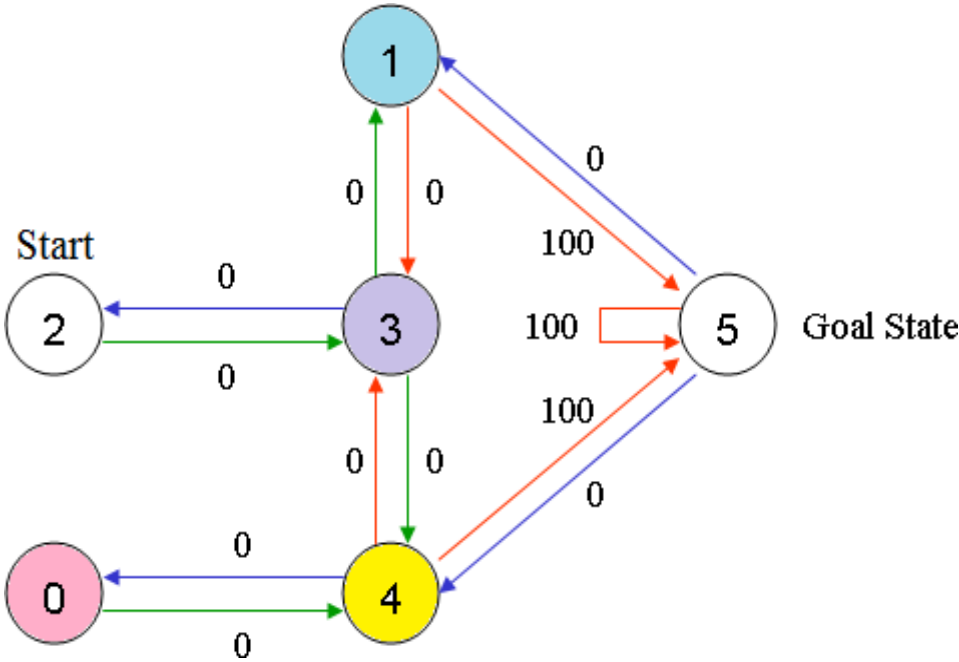
- Assign the Reward value to each door.
- The doors lead immediately to target is assigned an instant reward of 100.
- Other doors not directly connected to the target room have **zero** reward.
- For example, doors are two-way (0 leads to 4, and 4 leads back to 0), two edges are assigned to each room.
- Each edge contains an instant reward value



Understanding the Q – Learning

- Let consider agent starts from state *s* (Room) 2.
- Agent's movement from one state to another state is action *a*.
- Agent is traversing from state 2 to state 5 (Target).
  - Initial state = current state i.e. state 2
  - Transition State 2 → State 3
  - Transition State 3 → State (2, 1, 4)
  - Transition State 4 → State 5



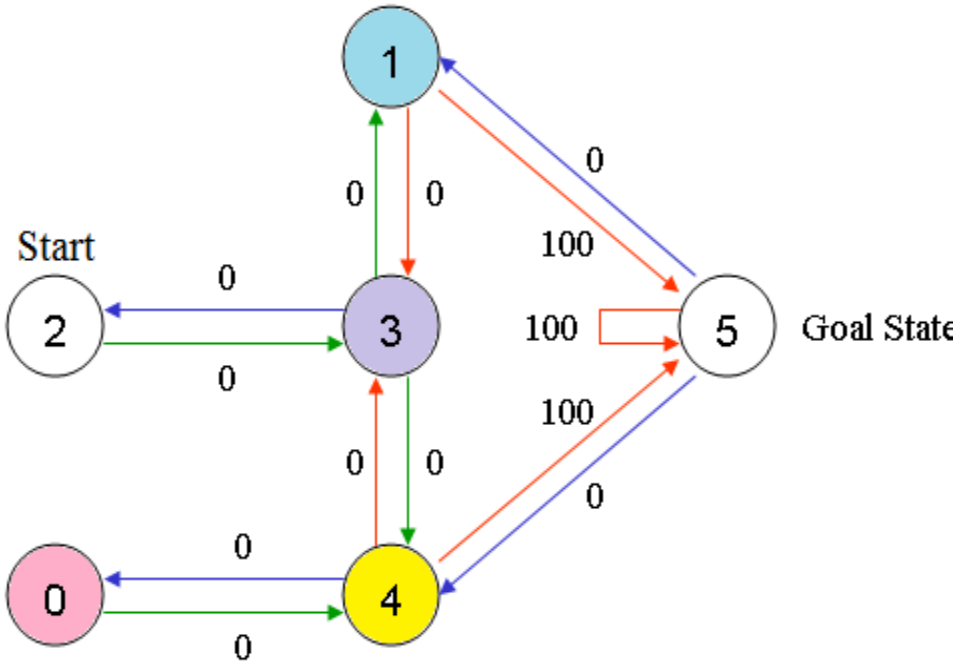


Understanding the Q – Learning

Prepare rewards table R (matrix)

- -1 denotes the no edge between the states
- 0 represents the indirect edge to the target

$$R = \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix}$$



Understanding the Q – Learning: **Prepare matrix Q**

- Matrix Q is the memory of the agent in which learned information from experience is stored.
- Row denotes the current state of the agent
- Column denotes the possible actions leading to the next state

Compute Q matrix:

*Q(state, action) = R(state, action) + Gamma * max[Q(next state, all actions)]*

- Gamma is discounting factor for future rewards. Its range is 0 to 1. i.e. 0 < Gamma <1.
- Future rewards are less valuable than current rewards so they must be discounted.
- If Gamma is closer to 0, the agent will tend to consider only the immediate rewards.
- If Gamma is closer to 1, the agent will tend to consider only future rewards with higher edge weights.

Q – Learning Algorithm

- Set the gamma parameter
- Set environment rewards in matrix R
- Initialize matrix Q as Zero
  - Select random initial (source) state
    - Set initial state *s* = current state
      - Select one action *a* among all possible actions using exploratory policy
    - Take this possible action *a*, going to the *next state s'*.
    - Observe reward *r*
  - Get maximum Q value to go to next state based on all possible actions
- Compute:
  - *Q(state, action) = R(state, action) + Gamma * max[Q(next state, all actions)]*
- Repeat the above steps until reach the goal state i.e current state = goal state

Example: Q – Learning

Matrix Q :

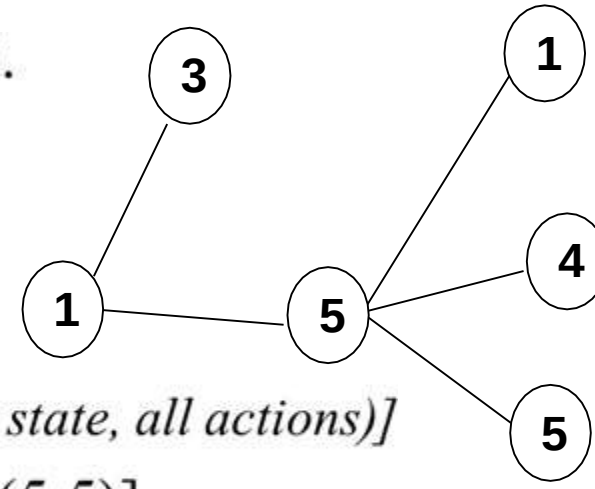
- Set the Gamma value = 0.8
- Initialize the matrix Q to zero matrix

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} \multicolumn{6}{c}{\text{Action}} \\ 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array}$$

Example: Q – Learning

- From state 1, agent can go to either state 3 or 5.
- Let's choose 5.
- From 5, Compute Max Q value to go to
- Next state based on all possible actions



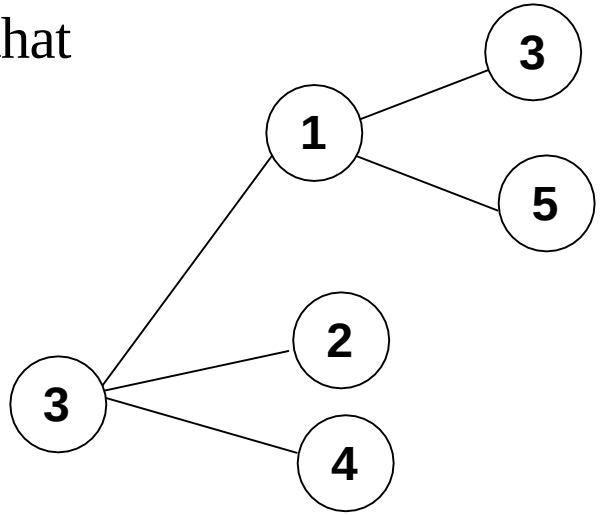
*Q(state, action) = R(state, action) + Gamma * max[Q(next state, all actions)]*

- $Q(1,5) = R(1,5) + 0.8 * \max[Q(5,1), Q(5,4), Q(5,5)]$
  $= 100 + 0.8 * \max[0, 0, 0] = 100 + 0 = \mathbf{100}$

- Update the Matrix Q.

$$R = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array} \quad Q = \begin{array}{c} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Action} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

- For next episode, choose next state 3 randomly that becomes current state.
- State 3 contains 3 choices i.e. state 1, 2 or 4.
- Let's choose state 1.
- Compute max Q value to go to next state based on all possible actions.



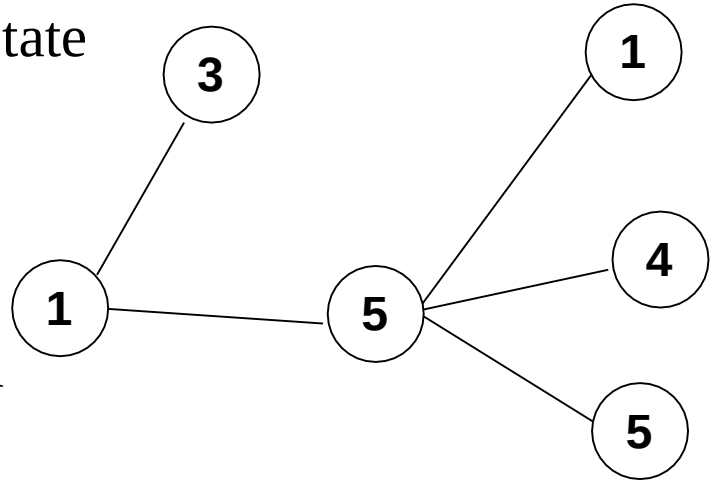
*Q(state, action) = R(state, action) + Gamma * max[Q(next state, all actions)]*

- Q(3,1) = R(3,1) + 0.8 * max[Q(1,3), Q(1,5)]
  = 0 + 0.8 * max[0, 100] = 0 + 80 = **80**
- Update the Matrix Q.

$$R = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array}$$

$$Q = \begin{array}{c} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Action} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

- For next episode, next state 1 becomes current state
- Repeat the inner loop due to 1 is not target state
- From State 1, either can go to 3 or 5.
- Let's choose state 5.
- Compute max Q value to go to **next state** based on all possible actions.



- *Q(state, action) = R(state, action) + Gamma * max[Q(next state, all actions)]*
- Q(1,5) = R(1,5) + 0.8 * max[Q(5,1), Q(5,4), Q(5,5)]
  = 100 + 0.8 * max[0, 0, 0] = 100 + 0 = **100**
- Q remains the same due to Q(1,5) is already fed into the agent. Stop process

$$R = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array} \quad Q = \begin{array}{c} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Action} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

References

- Tom Markiewicz& Josh Zheng,Getting started with Artificial Intelligence, Published by O'Reilly Media,2017
- Stuart J. Russell and Peter Norvig,Artificial Intelligence A Modern Approach
- Richard Szeliski, Computer Vision: Algorithms and Applications, Springer 2010